

Java Full Stack Development

1. Kafka Basics



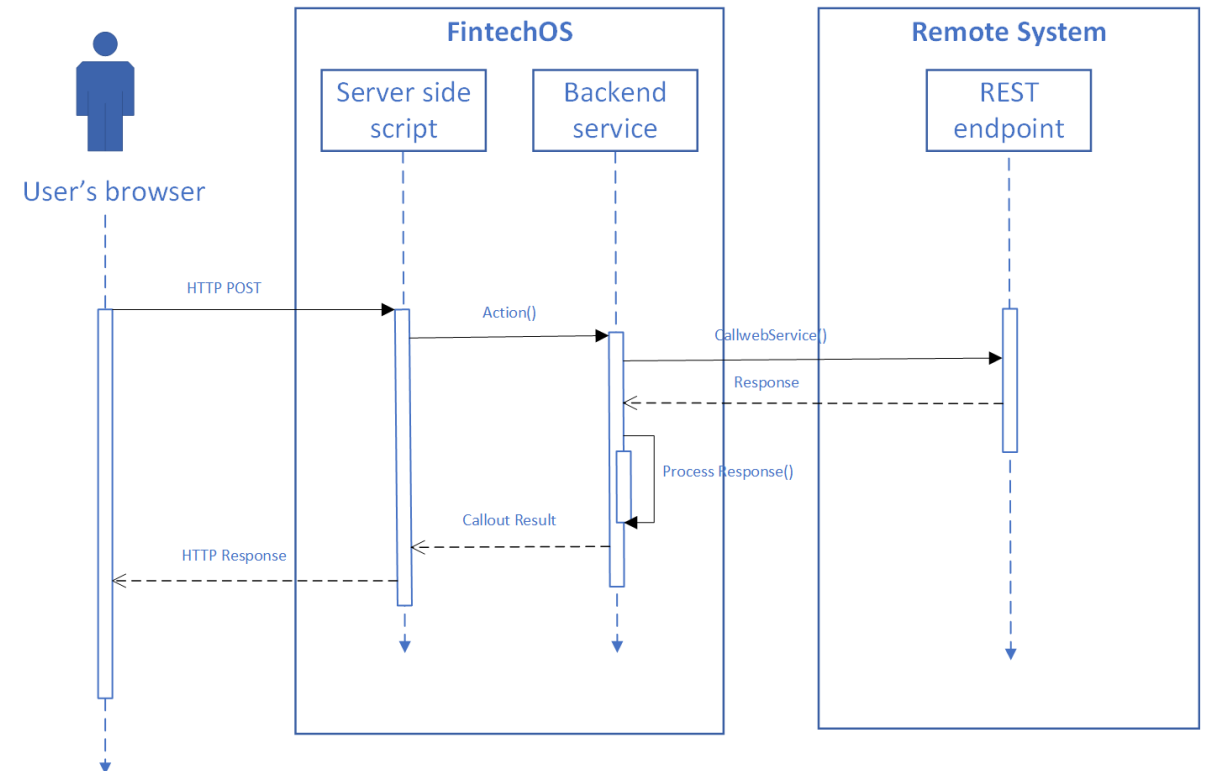
Module Topics

- Event Driven Architecture
- Core Kafka Concepts and Terminology
- Kafka Architecture
- Producers and Consumers
- Design Considerations

Event Driven Architecture

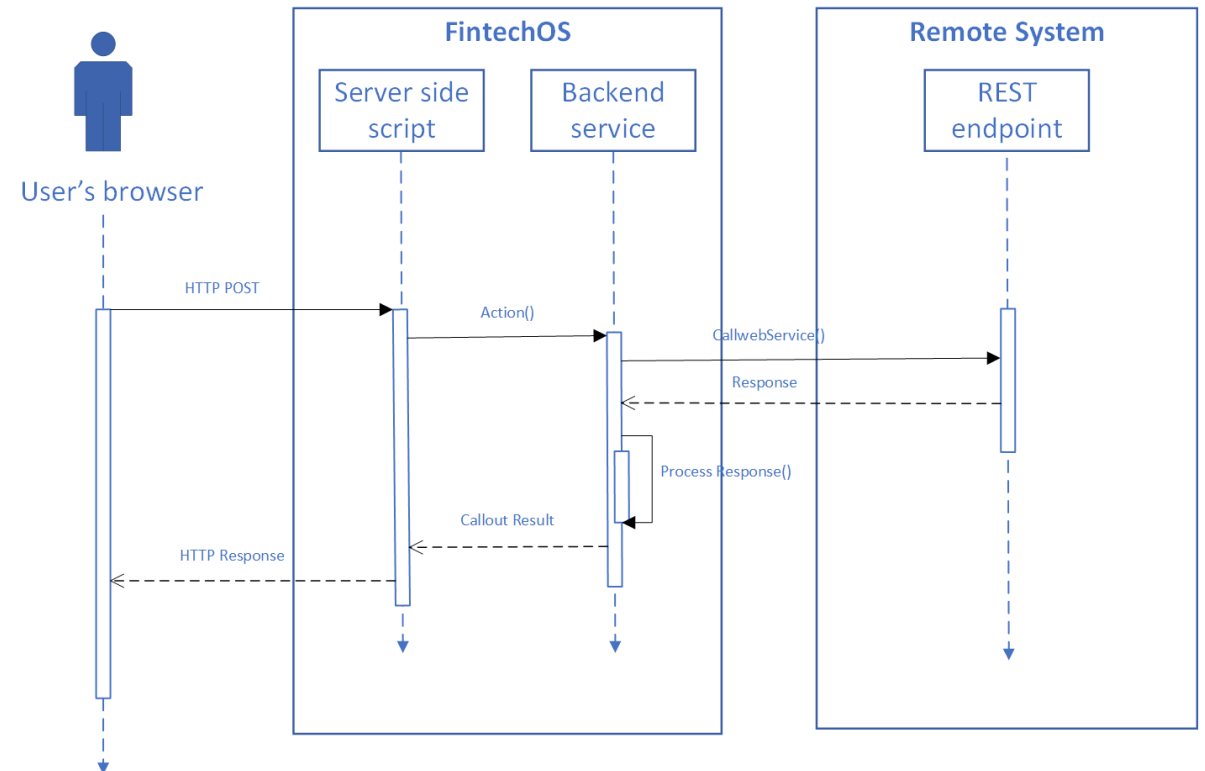
Synchronous Point-to-Point

- Service to service call and response
 - Service A calls Service B over HTTP and waits for a response
 - Simple, familiar, easy to reason about for a single call
- Implicit assumptions baked into this model:
 - B is reachable right now
 - B will respond quickly enough
 - B can handle A's current call rate
 - A has nothing better to do while waiting
- These assumptions hold in small systems
 - They erode rapidly as the system grows



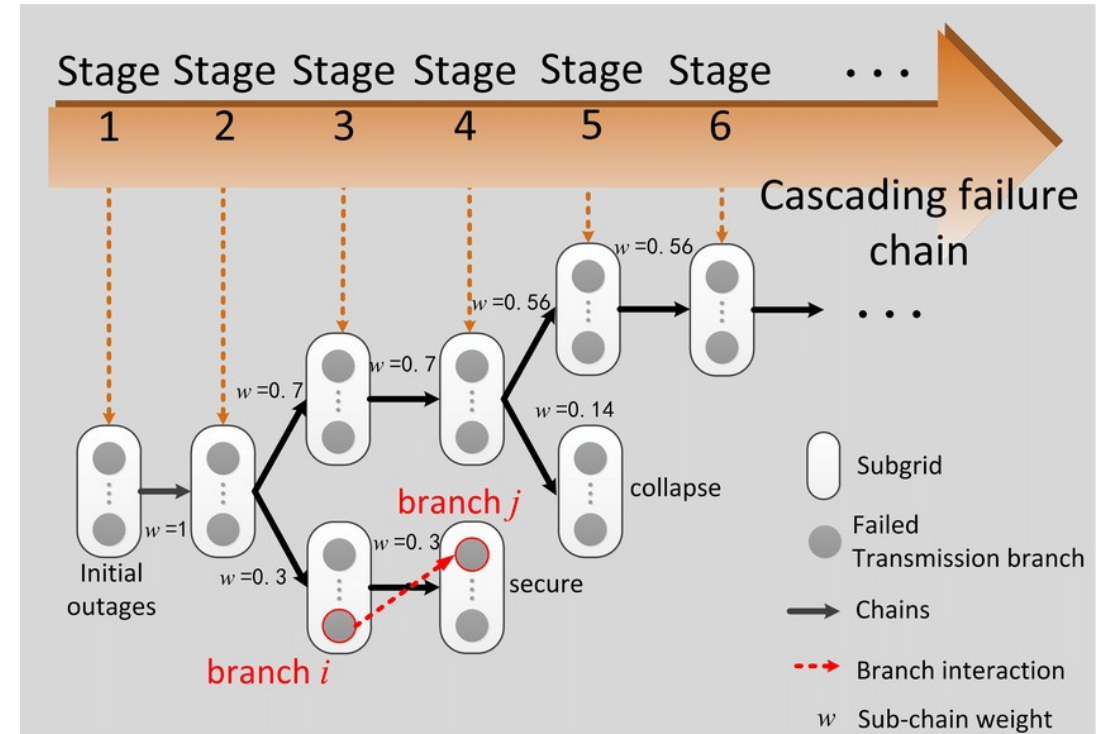
Temporal Coupling Problem

- Synchronous integration requires the caller and callee to be available simultaneously
 - If B is down, A cannot complete its work
 - If B is slow, A is slow
 - This is temporal coupling
 - Services are bound together in time
- Consequences
 - Deployments of B affect availability of A
 - Becomes operationally entangled with it under synchronous integration



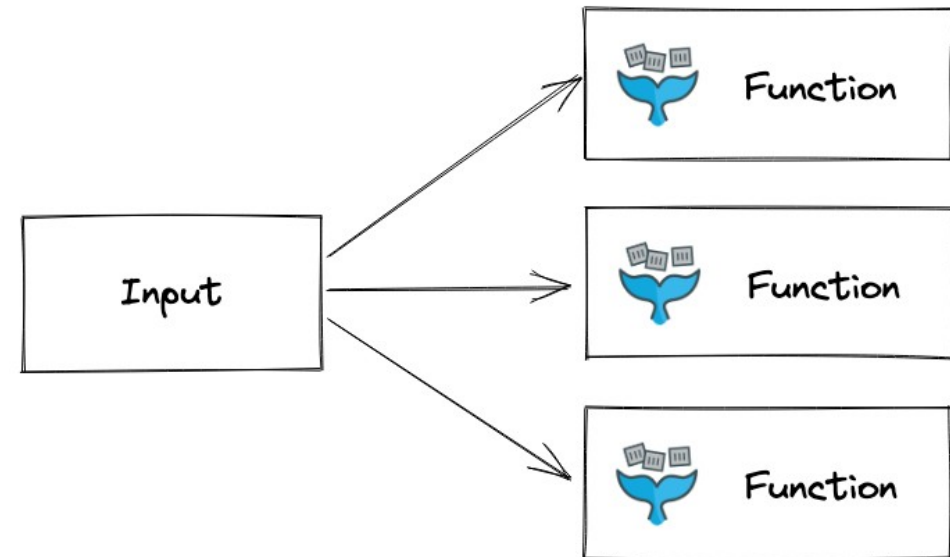
Cascading Failures Problem

- When one service fails, the chain fails
 - A calls B calls C calls D
 - D slows down $\rightarrow$ C's threads block $\rightarrow$ B's threads block $\rightarrow$ A's threads block
 - A single slow downstream dependency can exhaust resources up the entire chain
 - This is the cascading failure pattern
- Common mitigations
 - Timeouts to bound how a wait lasts
 - Circuit breakers to stop calling a known-bad service
 - Bulkheads to isolate resource pools
 - Mitigations only treat the effects of the architecture



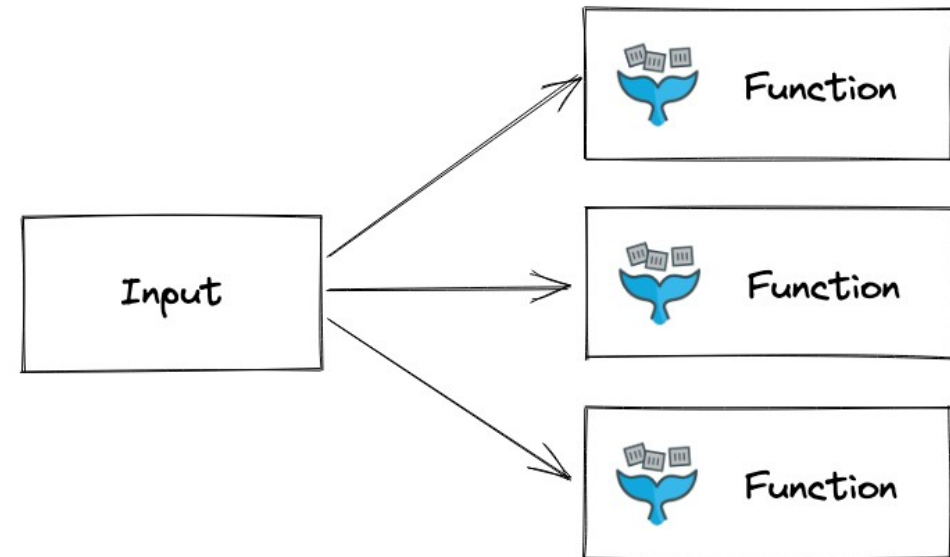
Fan-Out Scaling Failure

- A user updates their profile
 - The Profile service must now notify
 - Search (re-index)
 - Notifications (preference change)
 - Audit (compliance log)
 - Recommendations (refresh model)
 - Analytics (event capture)
 - With synchronous calls, Profile knows about all five consumers
 - Adding a sixth consumer requires changing Profile
 - Profile's latency is the sum of its dependents' latencies
 - Profile's availability is the product of its dependents' availabilities



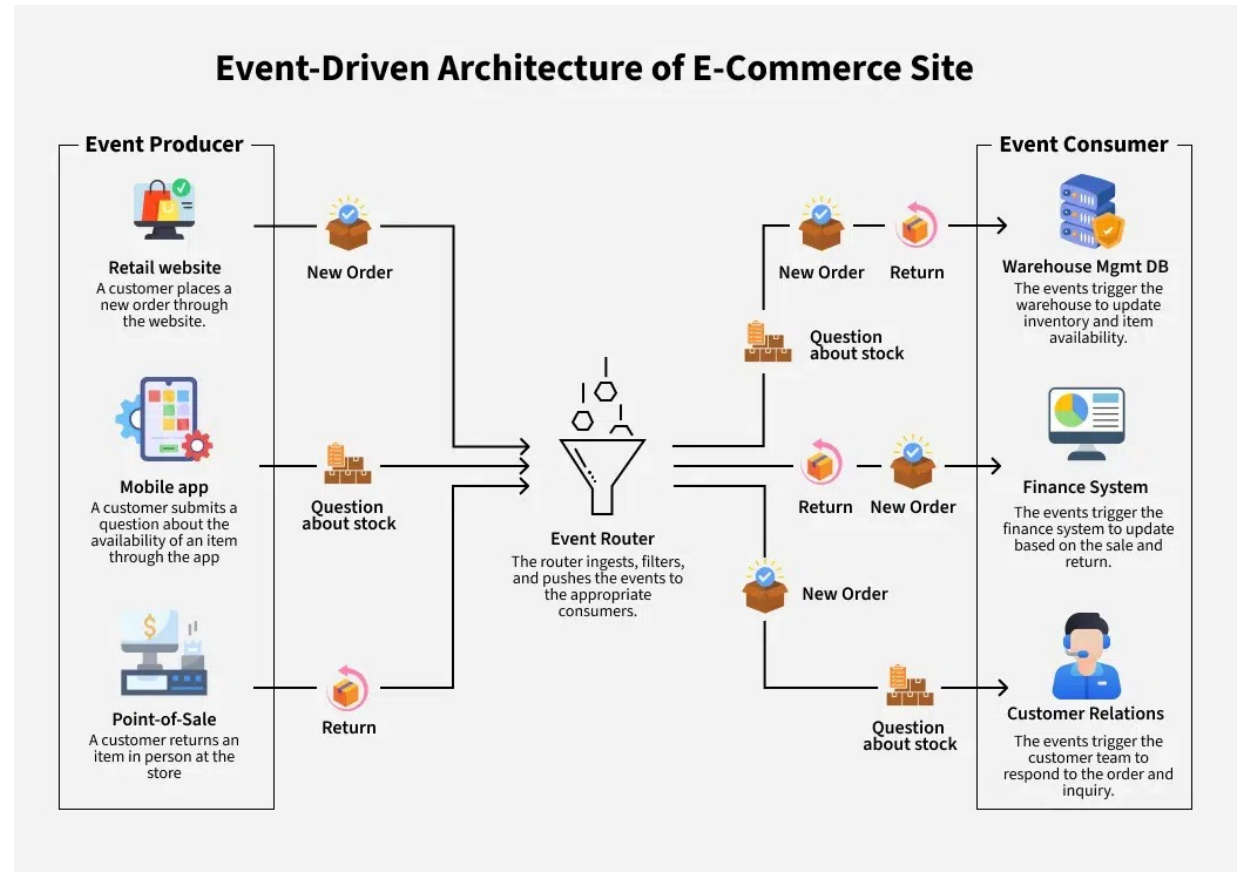
Fan-Out Scaling Failure

- The client becomes a coordinator
 - It has to know
 - Who cares about its events
 - Call each of them
 - Handle each of their failures
 - Pay the latency cost of all of them combined
 - Every new consumer is a code change to the producer
 - Violation of the open/closed principle at the architectural level



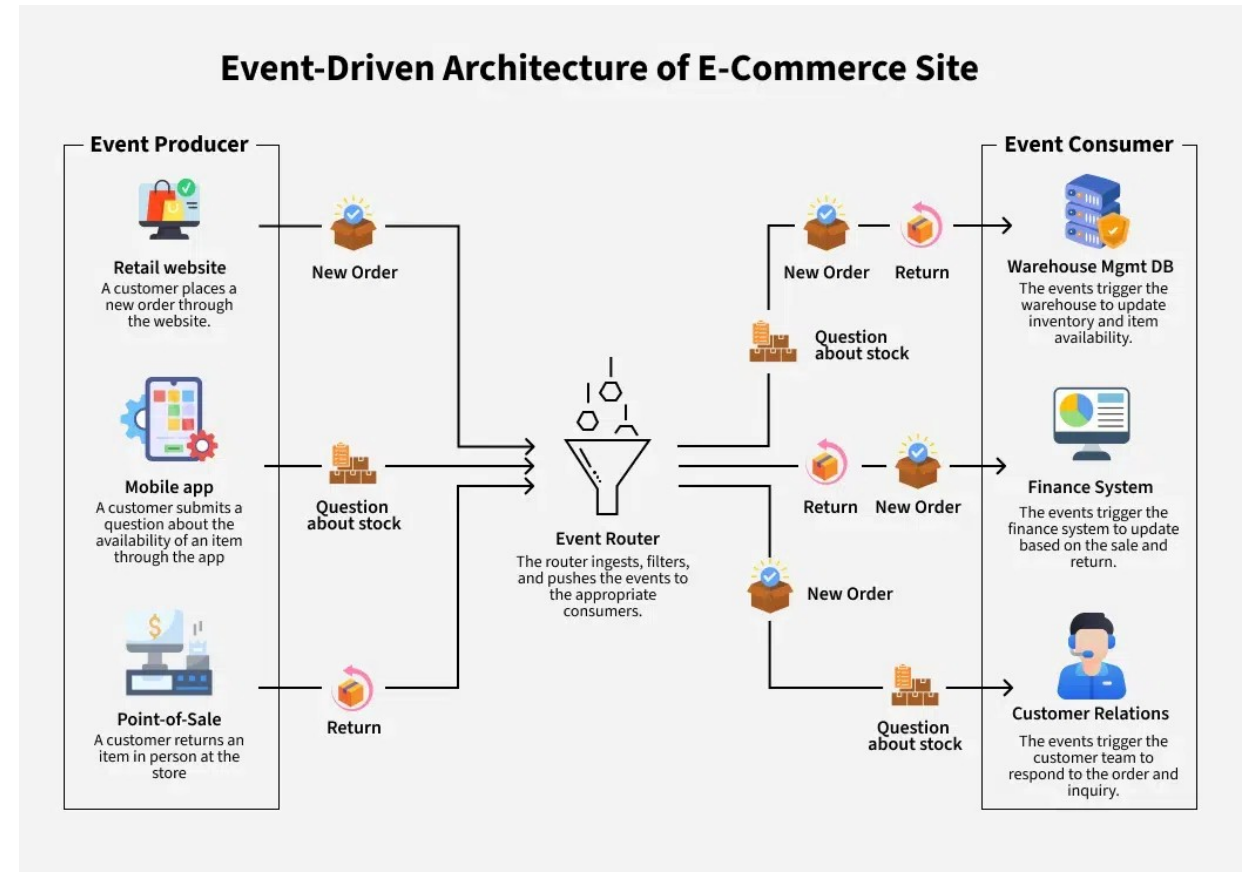
Even Driven Architecture

- In an event-driven system, the primary unit of integration is the event, not the call
 - An event = "something happened, here are the facts"
 - Producers emit events about their domain, without knowing who will consume them
 - Consumers subscribe to event streams, processing on their own schedule
- Communication is usually
 - Asynchronous: no waiting for a response
 - One-to-many: one event, many independent consumers
 - Decoupled in time: producer and consumer need not run simultaneously



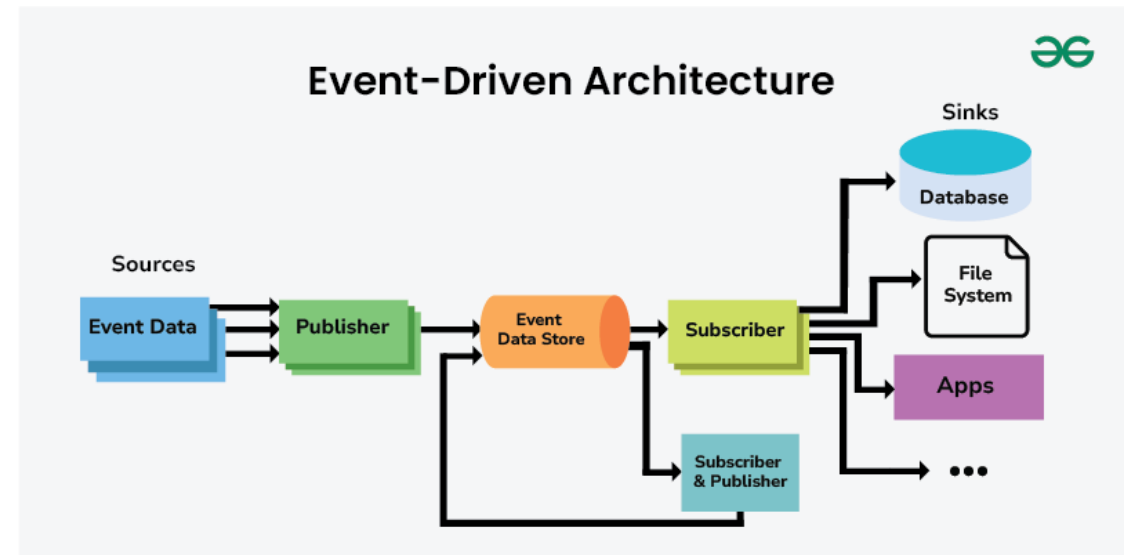
Even Driven Architecture

- In a request/response world
 - Services ask each other to do things
- In an event-driven world
 - Services announce what has happened in their domain, and other services react
 - The producer is no longer responsible for the downstream effects of its events
 - Its job is to faithfully publish what occurred
 - What anyone does with that information is their concern, not the producers



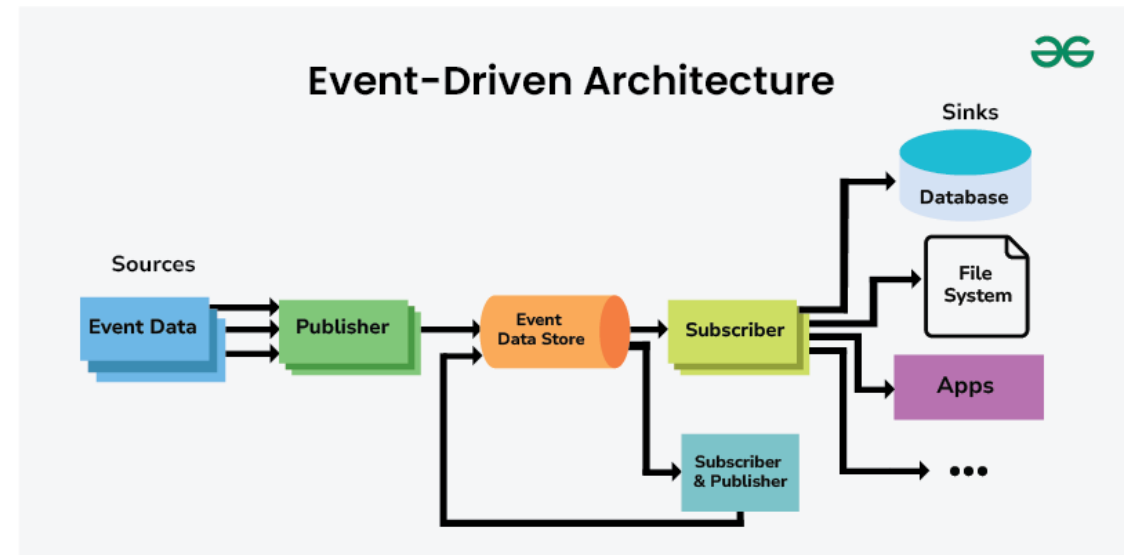
Even Driven Architecture

- Characteristics of event-driven systems
 - Loose coupling
 - Producers don't know consumers; consumers don't know producers
 - Temporal decoupling
 - Services run on independent schedules
 - Replayability
 - Events are durable; consumers can re-process history
 - Auditability
 - The event stream is a record of what happened
 - Extensibility
 - New consumers added without producer changes
 - Natural parallelism
 - Events partitioned across consumers



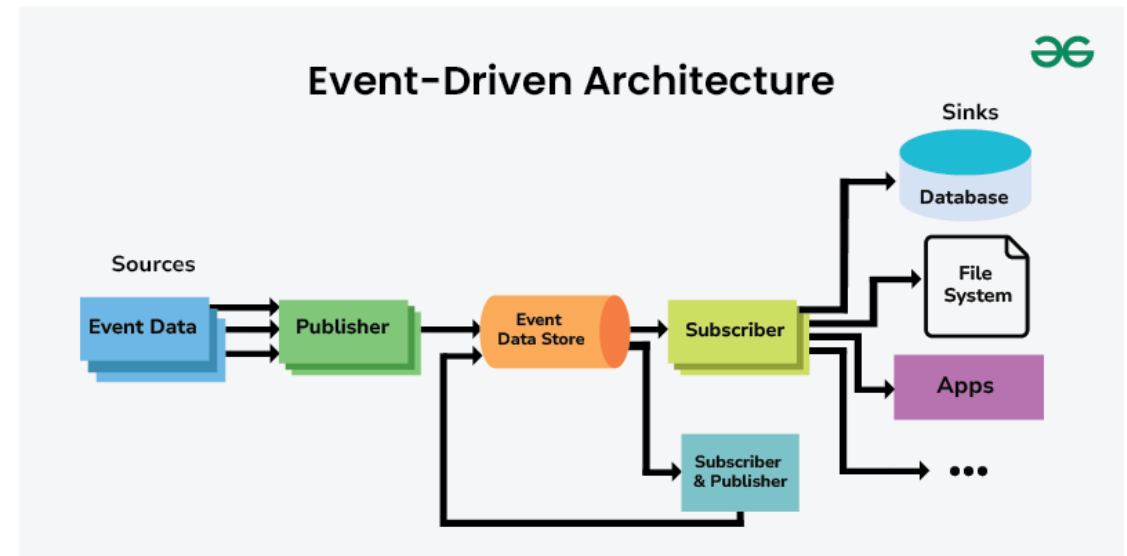
Even Driven Architecture

- Where it works well
 - Workflows where multiple services react to the same domain event
 - Long-running processes that span minutes, hours, or days
 - High-throughput data ingestion and processing pipelines
 - Systems that need an audit trail of state changes
 - Decoupling teams that need to release independently
 - Integration across organizational or team boundaries



Even Driven Architecture

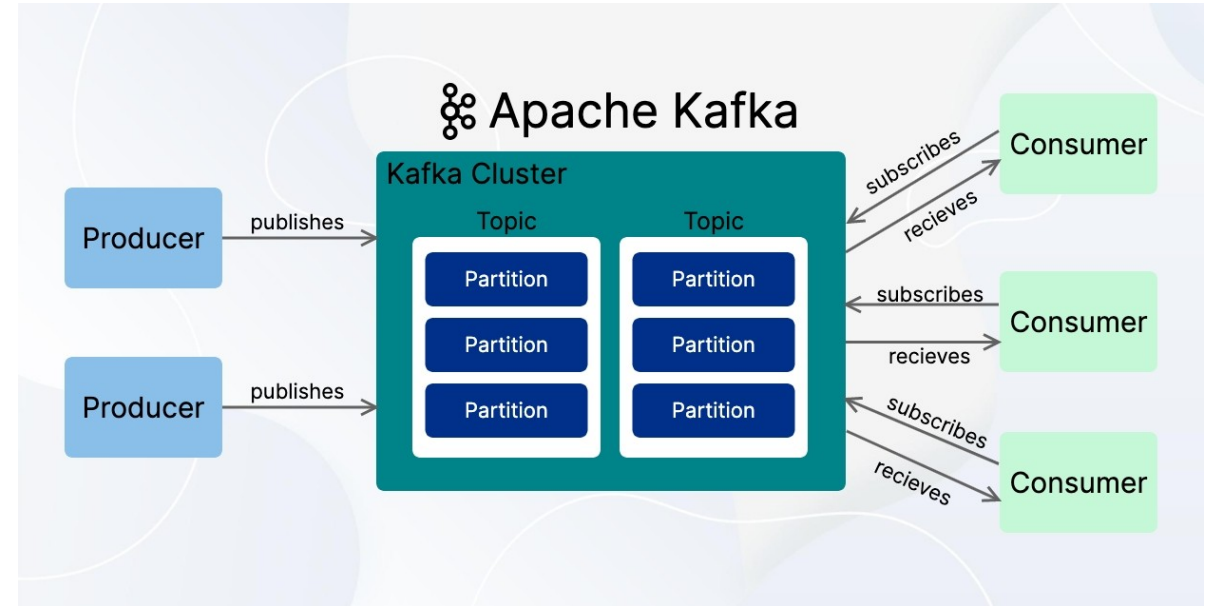
- Where it should not be used
 - Synchronous user-facing reads where the user is waiting for an answer
 - Strong-consistency operations that must succeed or fail atomically
 - Simple two-service integration with no fan-out
 - Workflows where the caller genuinely needs the result of the call
 - Operations where eventual consistency is unacceptable to the business



Core Kafka Concepts and Terminology

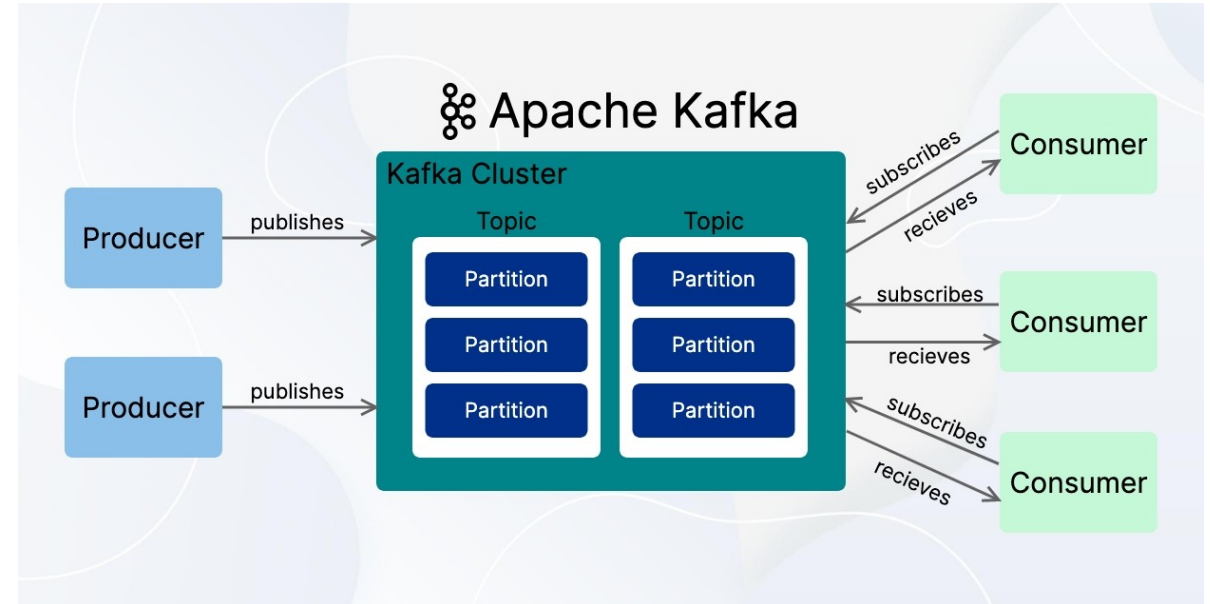
Kafka

- Kafka is a Publisher / Subscriber (Pub-Sub) messaging system
 - Distributed
 - Scales seamlessly
 - High throughput
 - Capable of handling billions of messages per day
 - Replicated
 - Safeguards data in case of machine failures
- Created by LinkedIn in 2010
 - Now Apache Project (Open Source)



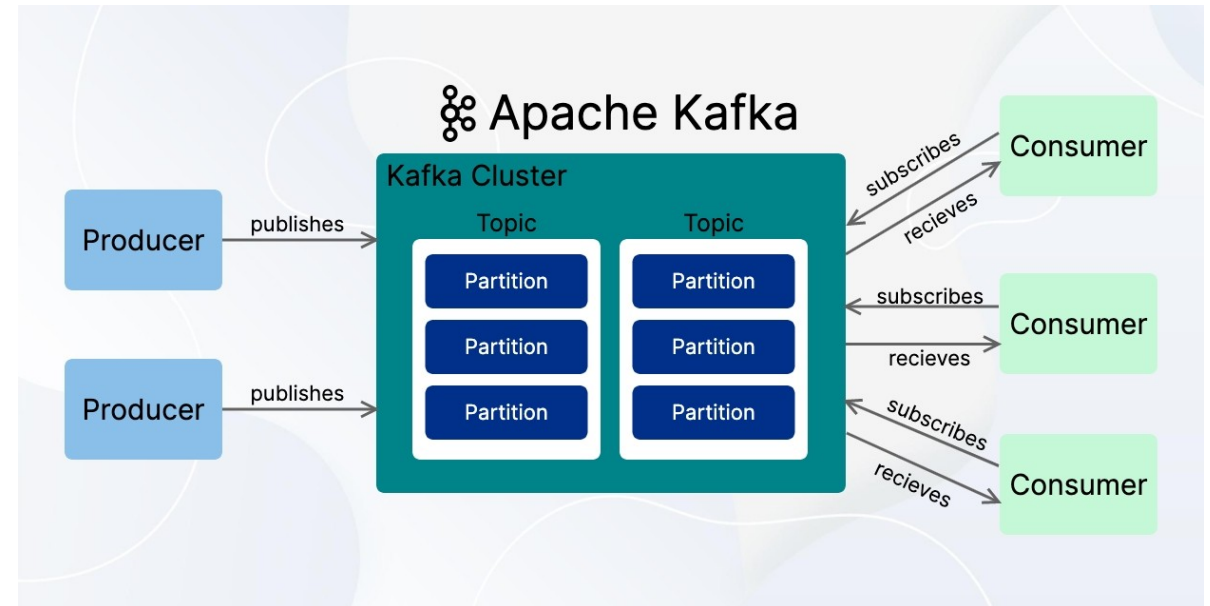
Kafka

- Why LinkedIn Built Kafka?
 - They had lots of databases built to store data
 - Piles of data: relational / key-value / caches / search indexes
- What was missing?
 - Something to handle the continuous flow of data
 - Existing message systems like ActiveMQ, were not able to handle the volume LinkedIn was seeing
 - This motivated LinkedIn to build Kafka from scratch
 - Why name it Kafka?
 - Kafka's co-creator Jay Kreps says "Kafka is optimized for writing" and named after author Franz Kafka



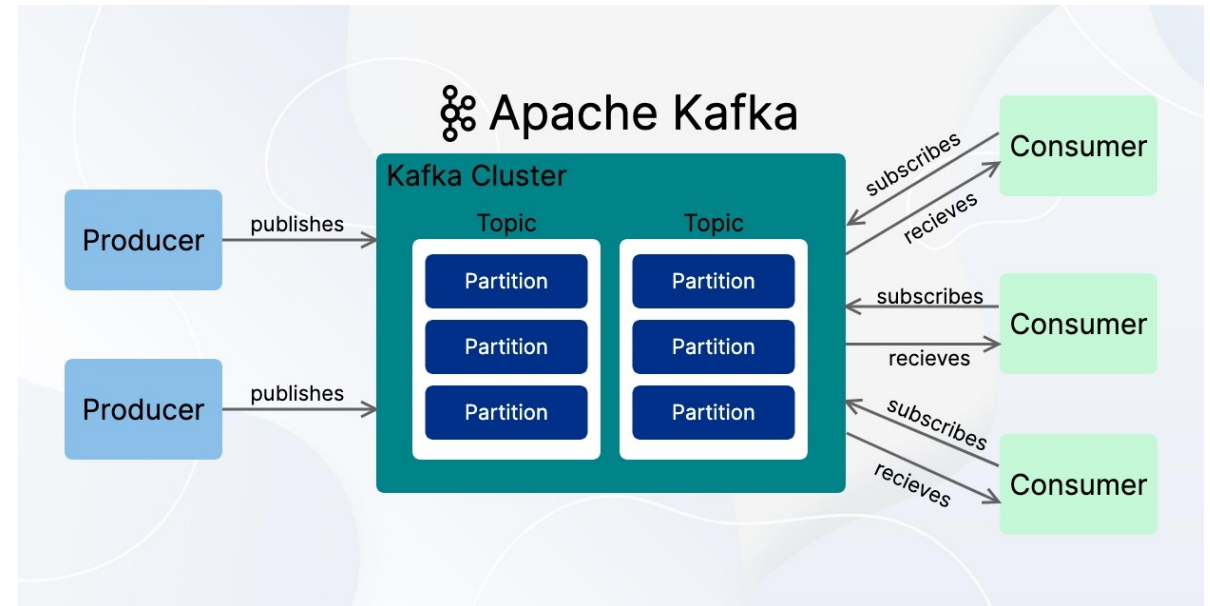
Kafka

- Message deletion
 - In most systems, after a message is 'read' it can be deleted
 - In Kafka
 - Messages are not deleted after they are consumed
 - There is client client API to delete a message
 - Messages are expired by Kafka automatically after a certain time
 - Reason
 - Many applications can subscribe to a topic
 - A client deleting a message can deprive other clients of their input



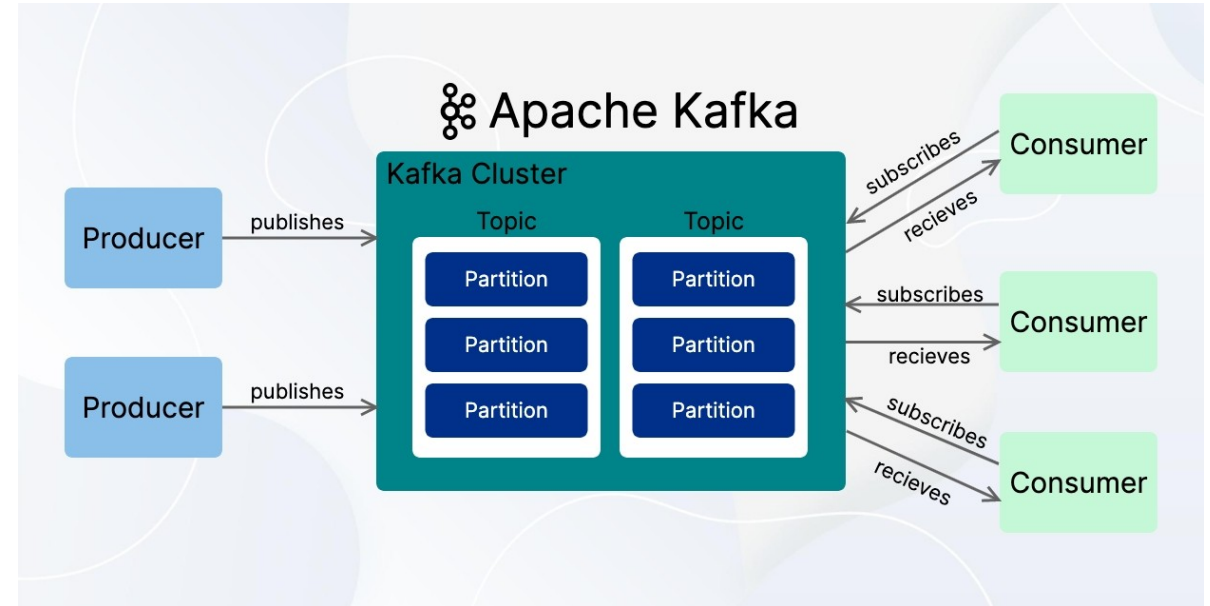
Kafka

- Message read order
 - Other message queues
 - Usually messages are read in order
 - FF and Rewind are not allowed operations
 - Kafka
 - Messages can be read in any order
 - FF and rewind are very fast operations
 - Reason
 - Clients can choose to skip messages if need to be



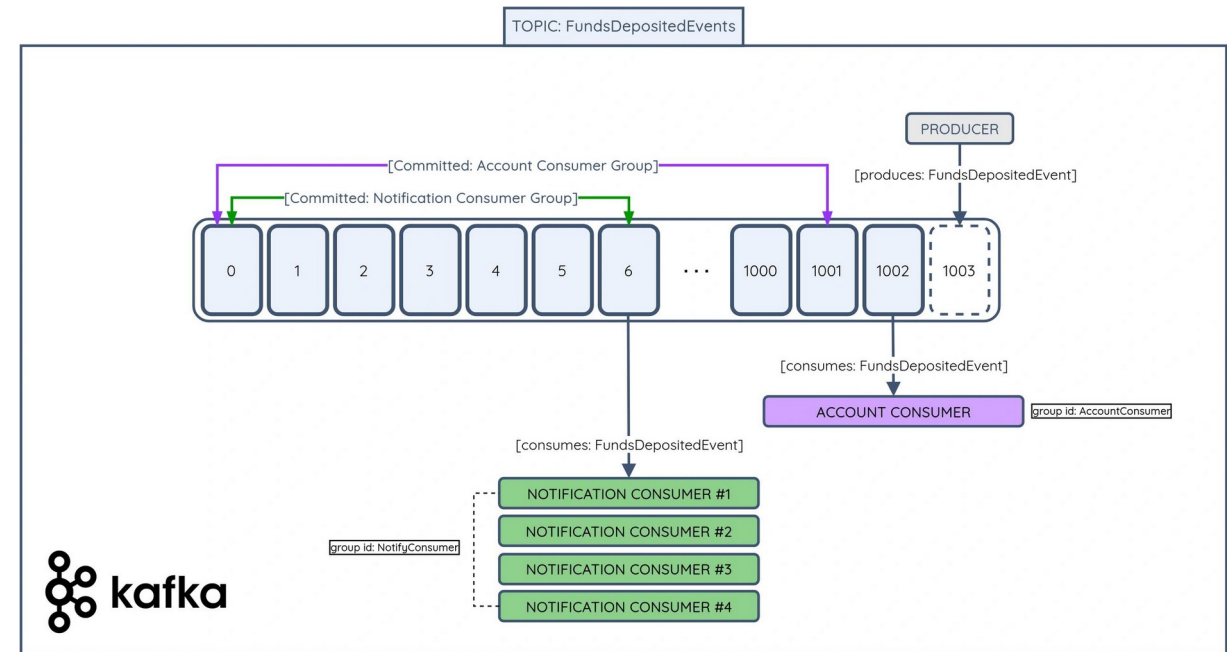
Kafka

- Message processing guarantee
 - Other message queues
 - Hard to guarantee one message will only be processed by one client
 - Kafka
 - Guarantees messages are sent to one consumer
 - No duplicate message processing
 - Uses the mechanism of consumer groups to facilitate this
 - Large number of clients can read / write to Kafka at high speed
 - No locking / blocking



Kafka

- Traditional message brokers
 - Think in terms of queues:
 - Messages arrive
 - Consumers take them
 - Messages disappear
- Kafka
 - Thinks in terms of an append-only log
 - Events are written to the end of a log file
 - Events stay there for a configured retention period
 - Consumers read from a position they choose
 - Many consumers can read the same log independently



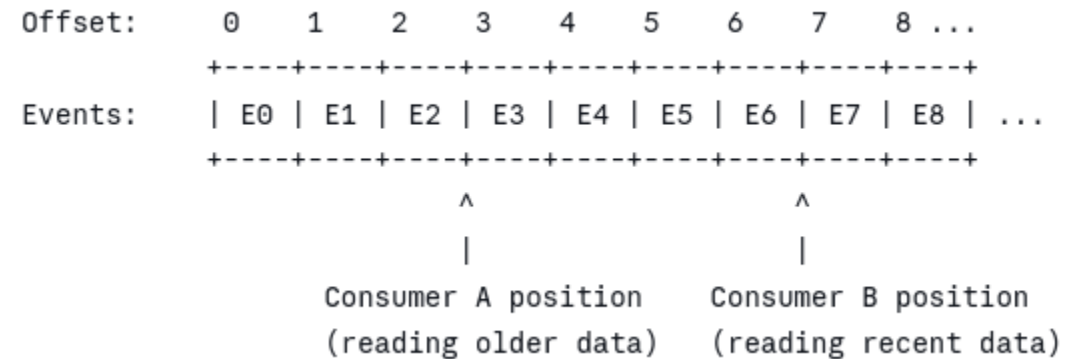
Use Cases

- Messaging and asynchronous service communication
 - The microservices integration use case
- Stream processing and real-time analytics
 - Continuous computation over event streams
- Event sourcing and durable system-of-record event streams
 - The event log is the source of truth
 - State is derived by replaying events

Property	Classical Broker (e.g., RabbitMQ queues)	Kafka
Data model	Queue of messages	Append-only log
Message lifetime	Until consumed	Configurable retention (hours to forever)
Consumer model	Push, broker tracks acks	Pull, consumer tracks offset
Multi-consumer	Requires fan-out exchanges	Native; each consumer group reads independently
Replay	Not supported natively	First-class capability
Throughput per node	Tens of thousands msg/sec	Hundreds of thousands to millions msg sec
Routing logic	Rich (exchanges, bindings)	Minimal (by topic/partition)

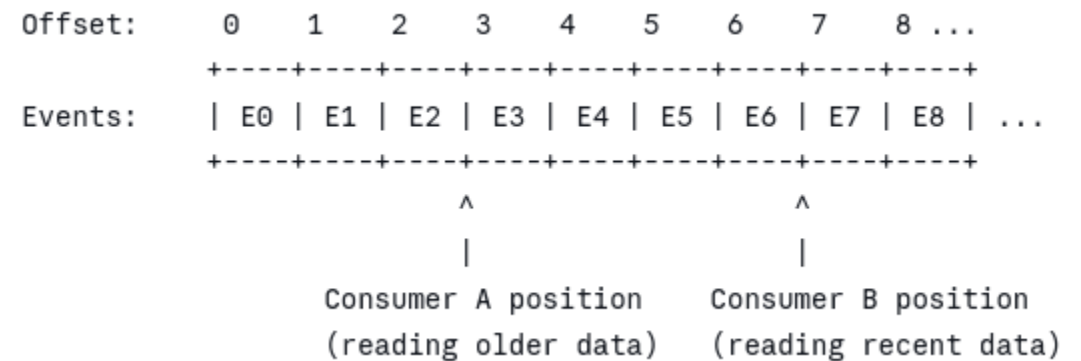
Visualizing the Log

- Events are appended to the right
 - Each event gets a permanent, increasing offset
 - Consumers track their own offset independently
 - No event is "consumed" in the traditional sense, it is read non-destructively



Visualizing the Log

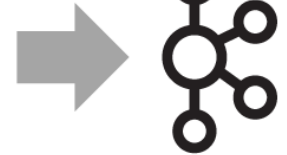
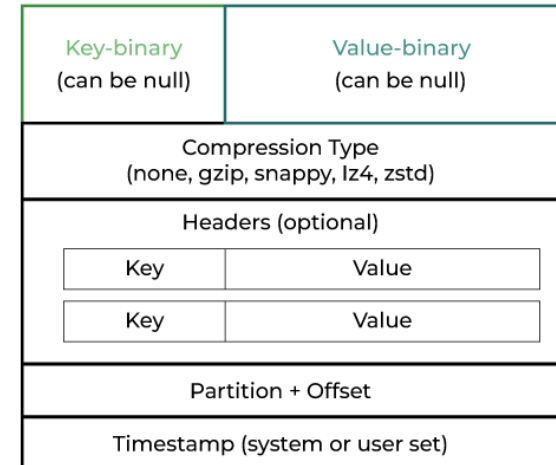
- A log is a totally ordered, append-only sequence of records
 - This simple structure underlies many distributed systems concepts:
 - Database transaction logs (write-ahead logs)
 - Replication protocols (Raft, Paxos use logs)
 - Filesystem journaling
 - Version control (Git is a log of commits)
 - Logs are easy to reason about because they are
 - Immutable: past entries don't change
 - Ordered: there is a clear "before" and "after"
 - Replayable: you can reconstruct state by reading from the start



Kafka Events/Messages

- Event
 - A single record written to or read from Kafka
 - Also called a record or message
- Anatomy of an event
 - Key (optional): used for partitioning and ordering
 - Value: the event payload (the "what happened")
 - Headers (optional): metadata key/value pairs
 - Timestamp: when the event occurred or was produced
 - Offset: assigned by the broker; the event's position in its partition

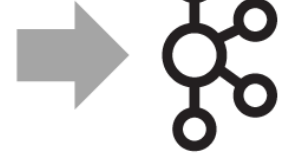
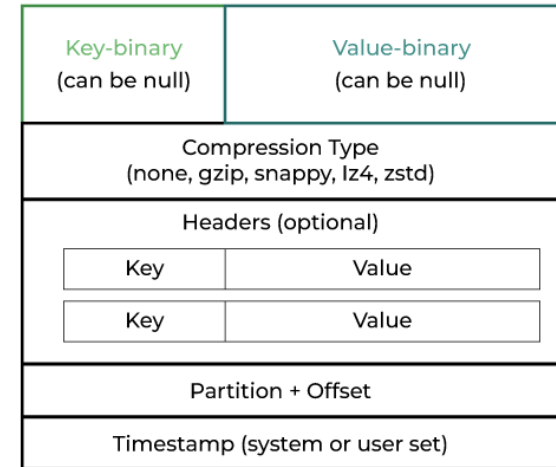
Kafka Message
created by
the producer



Kafka Events/Messages

- Event
 - Payload is a serialized byte string
 - Has no meaning for Kafka
 - Events are compressed by default
 - Headers provide handling metadata

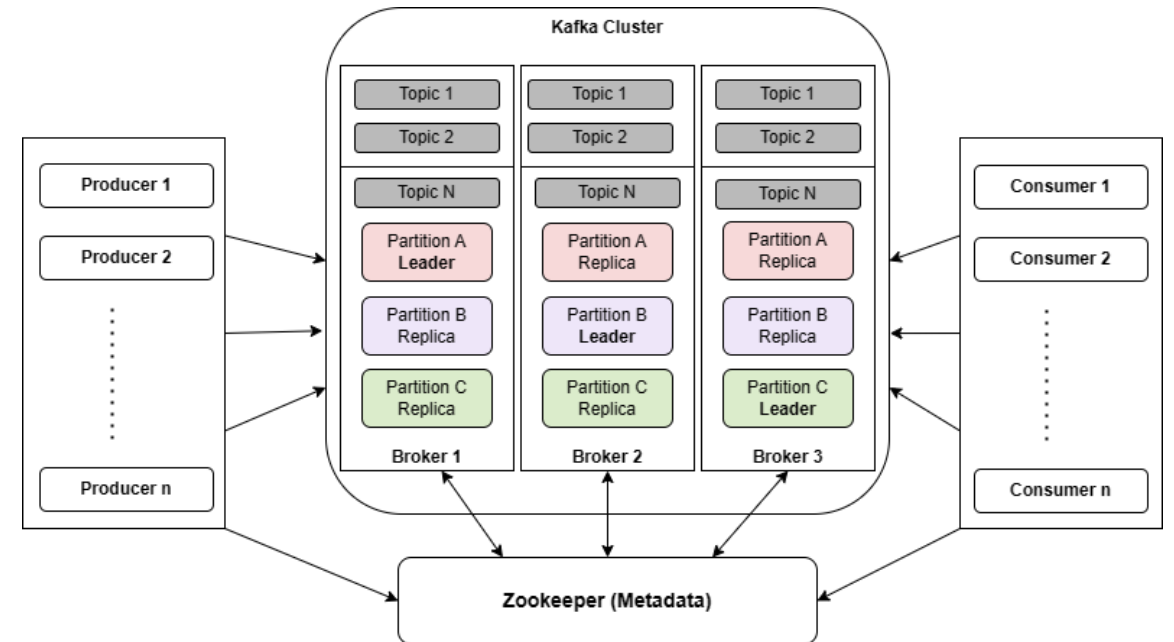
Kafka Message
created by
the producer



Kafka Architecture

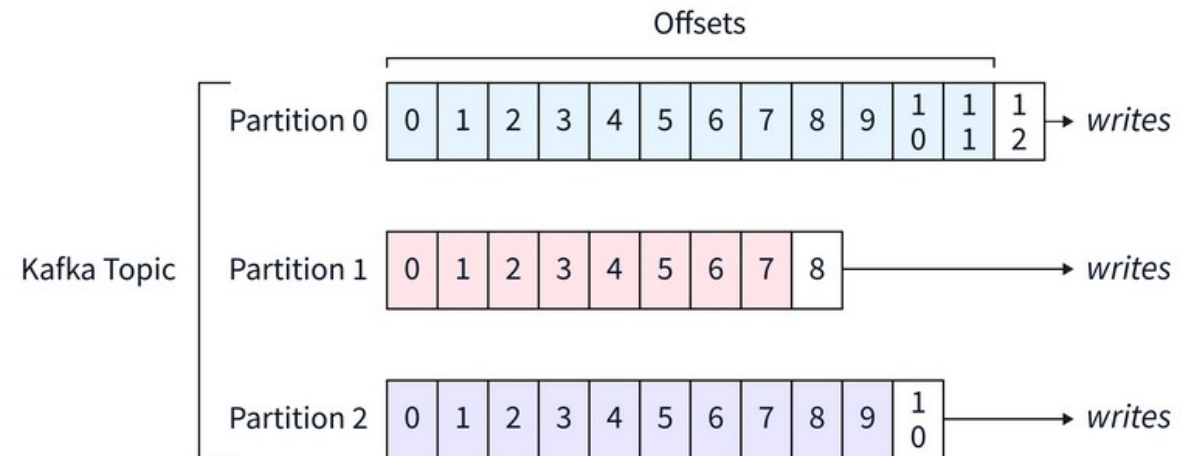
The Topic

- Topic
 - Named stream of related events
 - Examples
 - Orders.placed
 - users.profile-updated,
 - ayments.completed
 - Topics are:
 - Multi-producer: many services can write to the same topic
 - Multi-consumer: many services can read from the same topic
 - Persistent: events are stored on disk for the configured retention
 - A topic is a logical abstraction
 - Physically it is divided into partitions



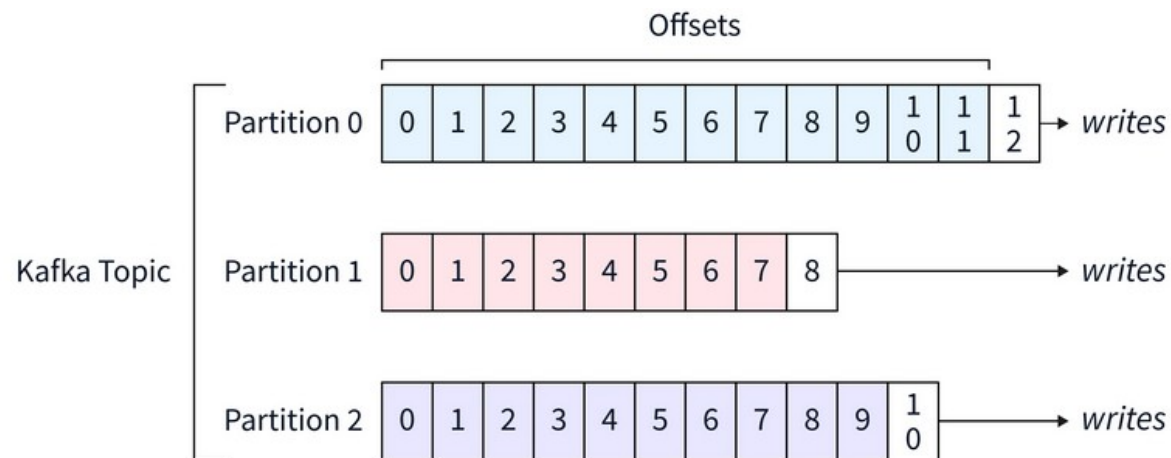
The Partition

- Each partition has its own offset sequence starting at 0
- An event's full address is the triple
 - (topic, partition, offset)
- Three partitions
 - Three independent logs that together form the topic



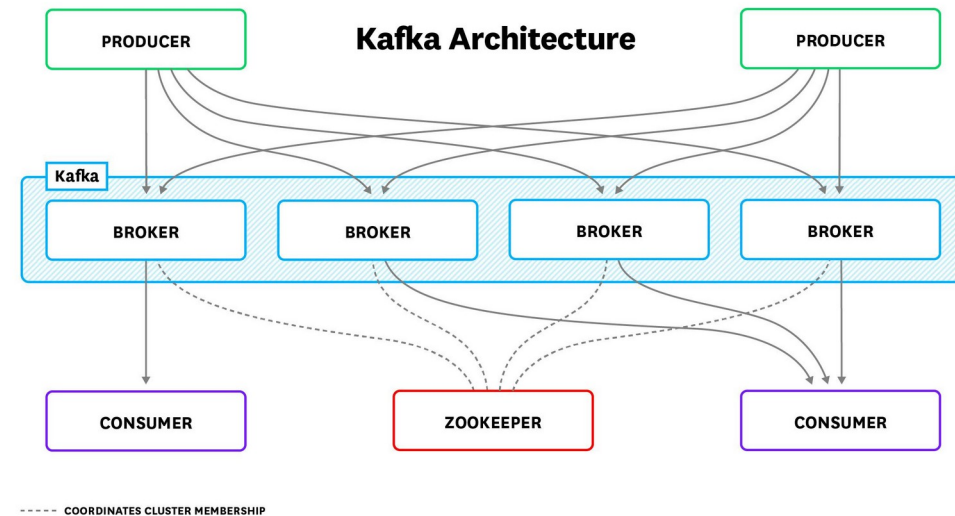
The Offset

- Permanent position marker
 - Monotonically increasing integer assigned by the broker
 - Each partition has its own offset sequence
 - Offsets are
 - Permanent: the offset of an event never changes
 - Per-partition: the same offset value in different partitions refers to different events
 - Consumer-tracked: each consumer remembers where it is in each partition
 - The offset is how a consumer answers: "Where am I in the log?"



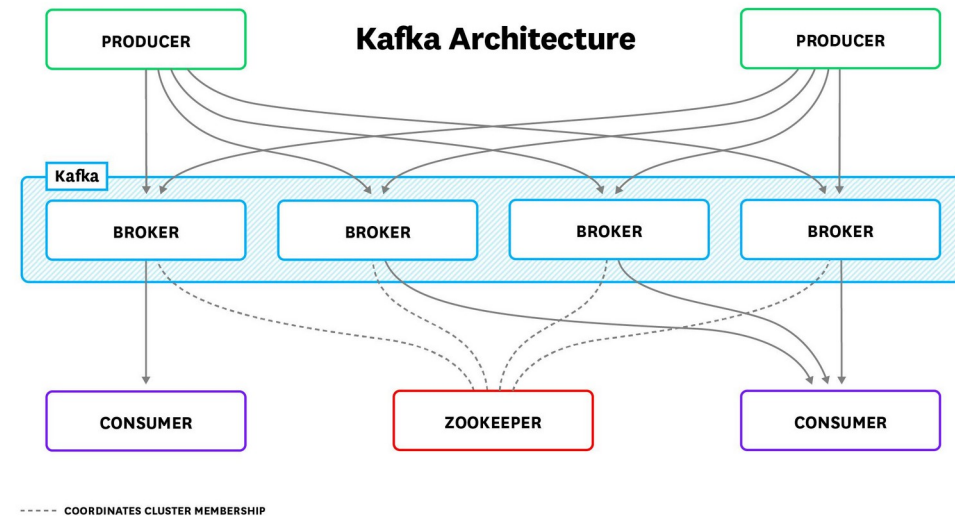
The Producer

- A client that writes events to Kafka topics
 - Producer responsibilities
 - Serialize the event (key and value) to bytes
 - Choose a partition explicitly, or implicitly
 - Send the event to the broker that leads that partition
 - Handle responses, retries, and failures
 - Producers are typically embedded in application services
 - A single application can have one producer that writes to many topics



The Consumer

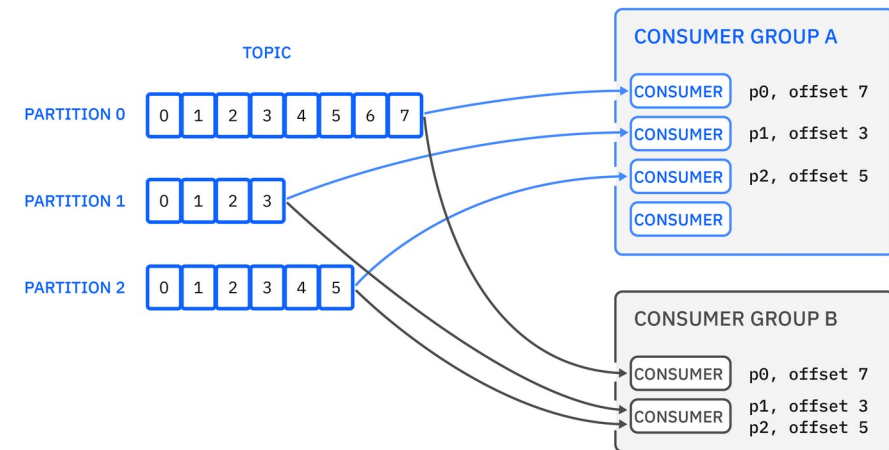
- A client that reads events from Kafka topics
 - Consumer responsibilities
 - Subscribe to one or more topics
 - Receive partition assignments from the broker
 - Poll for events from assigned partitions
 - Process events and commit offsets
 - Consumers pull events at their own rate
 - The pull model means the consumer controls its own throughput



The Consumer Group

- A set of consumers that cooperate to read a topic
 - Kafka assigns each partition to exactly one consumer in the group
 - Consequences
 - Adding consumers to a group creates more parallelism up to partition count
 - Each event is processed by one consumer in the group
 - Multiple groups can read the same topic independently
 - Prevents consumers in the group from reading the same message twice
 - This is how Kafka delivers both load balancing and fan-out

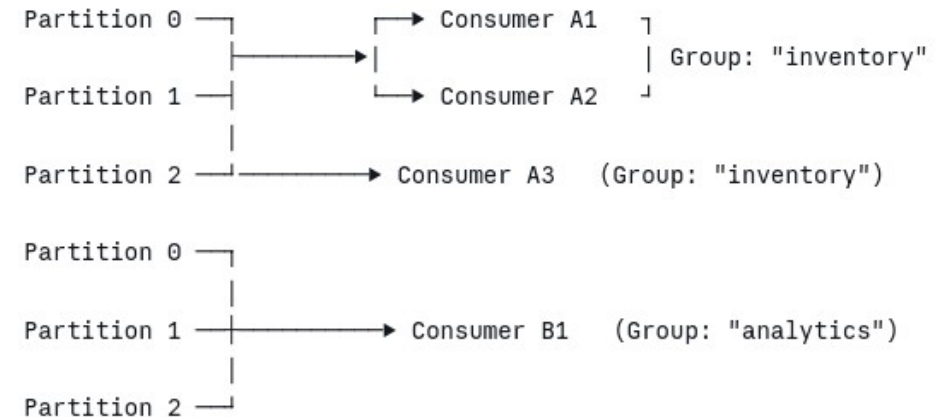
Consumer Groups



The Consumer Group

- Example:
 - Group “inventory” starts with two consumers registered with Kafka
 - A1 reads partitions 0 and 2
 - A2 reads partition 1
 - Kafka assigns partitions
 - A3 is added, partitions re-balanced
 - Partition 2 is reassigned to consumer A3
 - Consumer B1 has to read from all three partitions
 - Kafka ensures that all partitions in a topic are read by the consumers in a group
 - Avoids the problem of unread messages

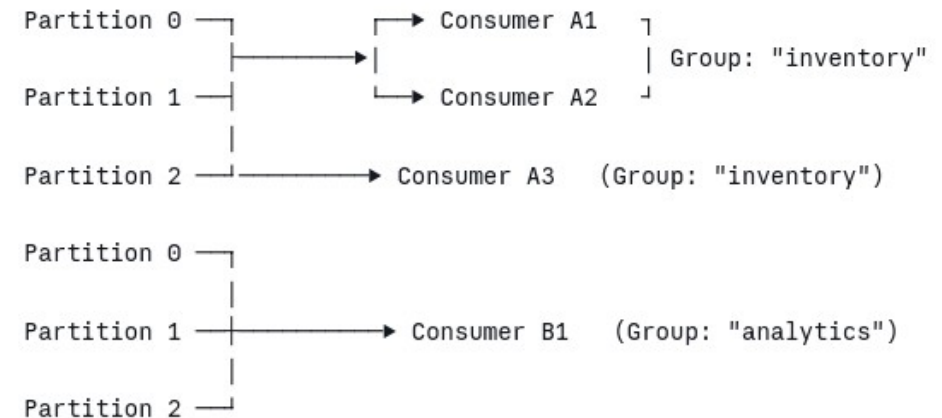
Topic: orders.placed (3 partitions)



Retention

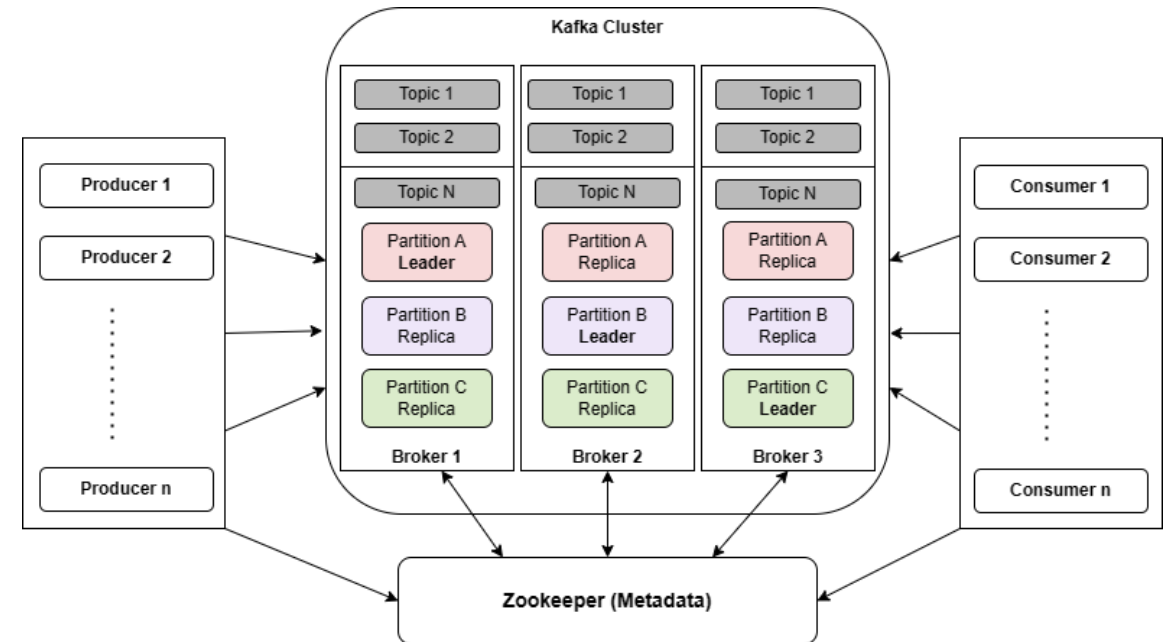
- Kafka does not delete events when consumed
 - Events are deleted based on retention policy, configured per topic
 - Two main retention strategies:
 - Time-based retention: keep events for N hours/days (default: 7 days)
 - Size-based retention: keep up to N bytes per partition
 - Whichever limit hits first triggers deletion
 - A third option is log compaction
 - Keep the latest event for each key, indefinitely
 - Useful when the topic represents "current state per entity"

Topic: orders.placed (3 partitions)



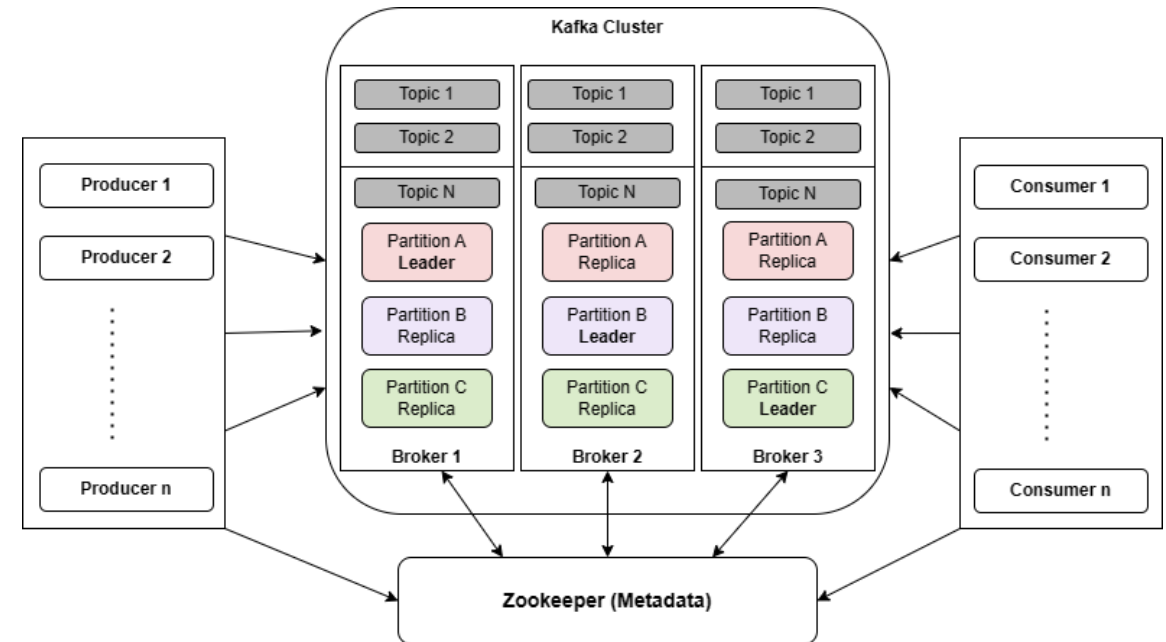
The Broker

- A broker is a single Kafka server process
 - Each broker
 - Hosts a subset of the cluster's partitions
 - Serves read and write requests for those partitions
 - Replicates partitions to and from other brokers
 - A broker is typically one machine
 - Either physical or virtual
- A production cluster has many brokers
 - Typically 3 to dozens or hundreds
 - Cluster metadata is managed by Zookeeper
 - There is no controller “node”
 - One broker is designated arbitrarily as the leader



The Cluster

- A cluster is a set of brokers operating together
 - Each partition has a configurable replication factor
- For each partition
 - One broker holds the leader replica that handles all reads and writes
 - Other brokers hold follower replicas
 - These are kept in sync with the leader
 - If a broker where the leader is stored fails
 - One of the followers is promoted to leader
 - This is what gives Kafka its durability guarantee



The Distributed Log

- A partition
 - Is a directory on the broker's filesystem
 - Inside the directory, the log is split into segment files
 - Only one segment is active, meaning being written to at a time
 - Older segments are immutable
 - They are never modified, only deleted
 - Each segment has companion index files for efficient lookup

```
orders.placed-3/
├─ 00000000000000000000000000000000.log    (old segment, closed)
├─ 00000000000000000000000000000000.index
├─ 00000000000000000000050000000000.log    (old segment, closed)
├─ 00000000000000000000050000000000.index
├─ 0000000000000000000098470000.log        (active segment)
└─ 0000000000000000000098470000.index
```

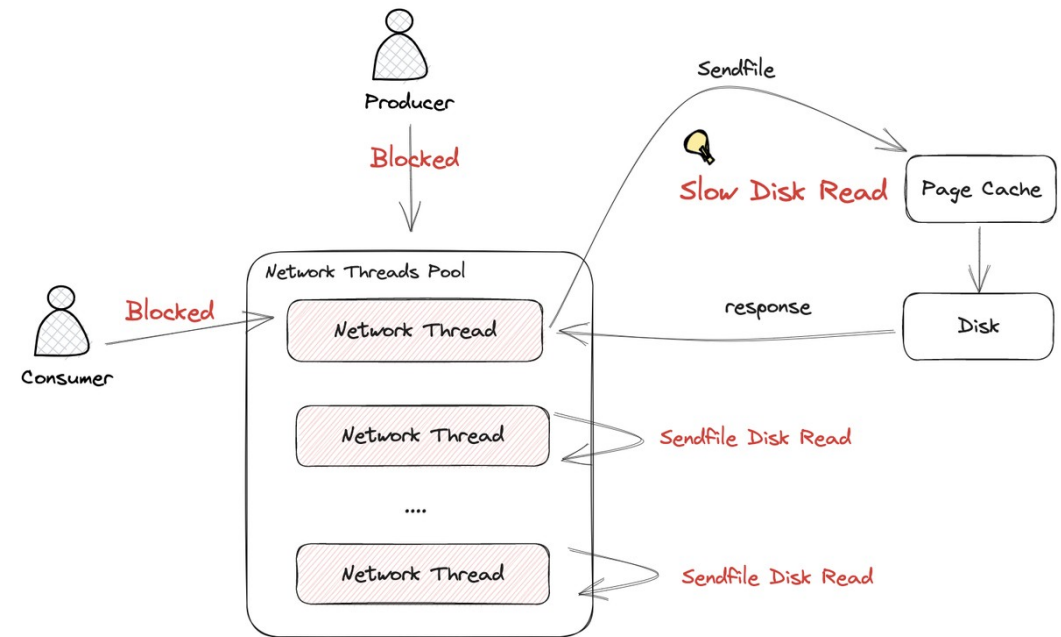
Performance

- Traditional databases require random disk I/O:
 - B-tree updates touch scattered pages
 - Random seeks are slow on any storage medium
- Kafka only ever appends to the active segment:
 - Writes are sequential
 - Sequential disk I/O is orders of magnitude faster than random I/O
 - On modern SSDs: random is faster, but sequential is still faster
 - On spinning disks: the difference was dramatic
 - Reads are also largely sequential
 - Consumers scan forward from an offset

```
orders.placed-3/  
├─ 000000000000000000000000.log      (old segment, closed)  
├─ 000000000000000000000000.index  
├─ 0000000000000050000000.log          (old segment, closed)  
├─ 0000000000000050000000.index  
├─ 000000000000009847000.log           (active segment)  
└─ 000000000000009847000.index
```

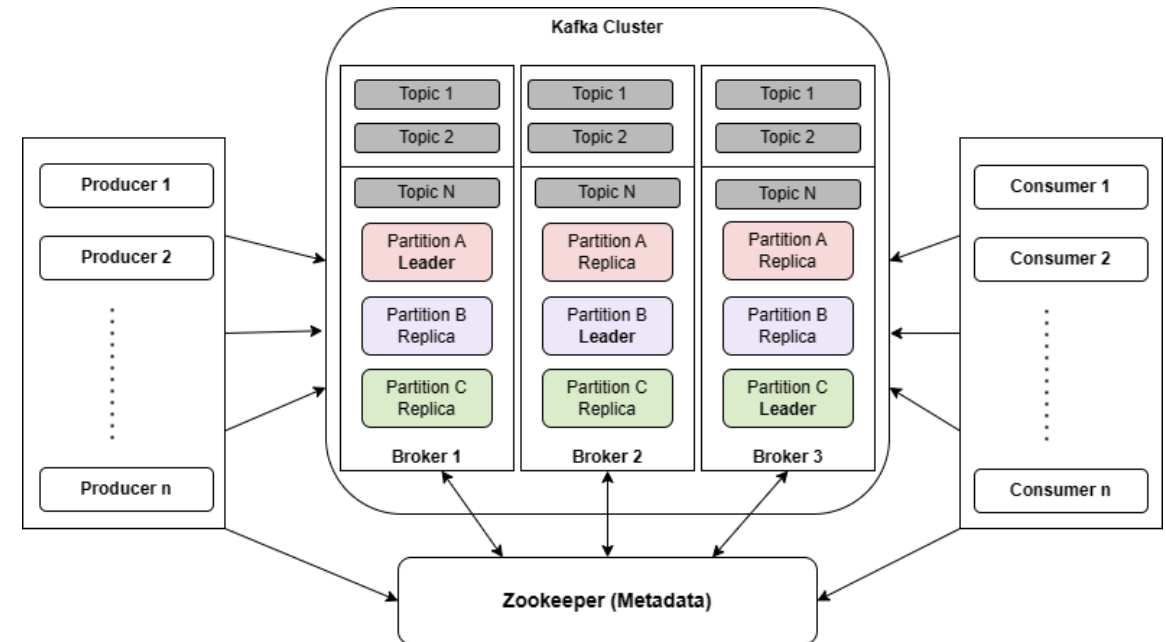
Page Cache

- Kafka
 - Does not maintain an in-memory cache of events
 - Relies on the operating system's page cache
 - When events are written
 - They land in the page cache first, then flush to disk
 - When consumers read recent events
 - The data is often still in the page cache: no disk I/O
 - Benefits:
 - No duplicate caching
 - Cache survives broker restarts because the page cache is a kernel structure
 - The OS has excellent cache management algorithms



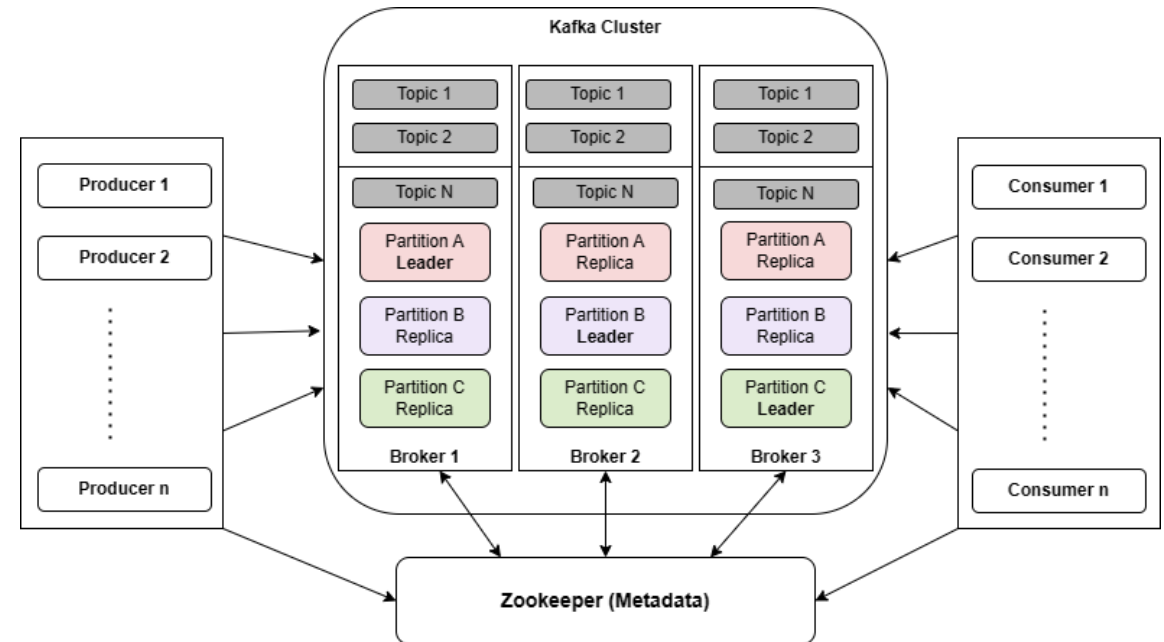
The Broker

- Broker's responsibilities
 - Host a subset of the cluster's partitions
 - Process producer write requests for partitions it leads
 - Process consumer fetch requests for partitions it leads
 - Replicate data to and from other brokers for partitions it follows
 - Report health and metadata to the cluster controller
 - Participate in leader elections when other brokers fail



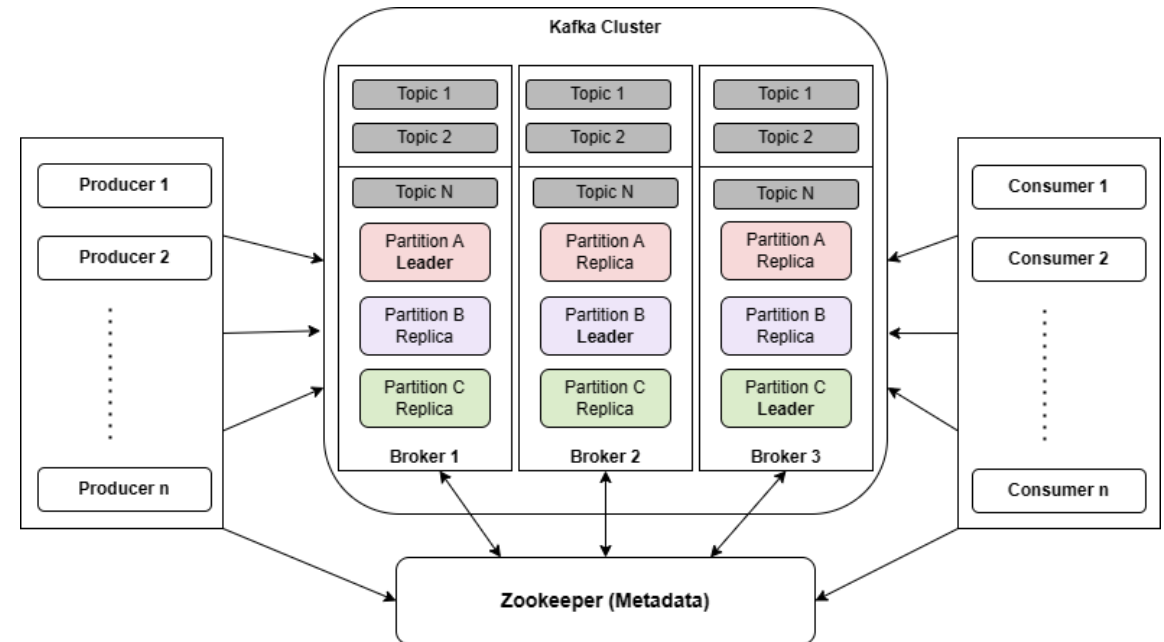
The Cluster

- Brokers presented as one service
 - Clients connect to any broker to discover cluster metadata
 - Metadata includes:
 - List of brokers
 - List of topics and partitions
 - Which broker is the leader for each partition
 - Clients then connect directly to the relevant broker for each partition
 - From the client's perspective, the cluster is a single logical service
 - Internally, the cluster is a coordinated set of peers



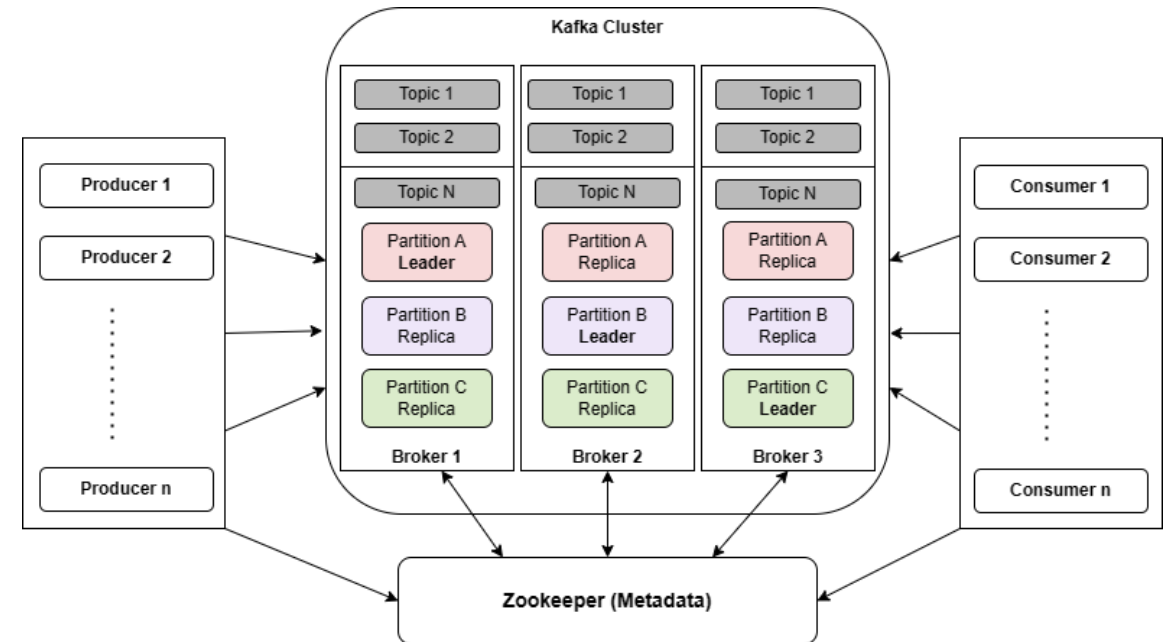
The Controller

- A single broker elected to coordinate cluster-wide decisions
 - Controller responsibilities:
 - Track broker liveness (who is up, who is down)
 - Manage partition leader assignments
 - Trigger leader elections when needed
 - Handle topic creation, deletion, and configuration changes
 - Every Kafka cluster has exactly one controller at a time
 - If the controller dies, another broker is elected to replace it



ZooKeeper and KRaft

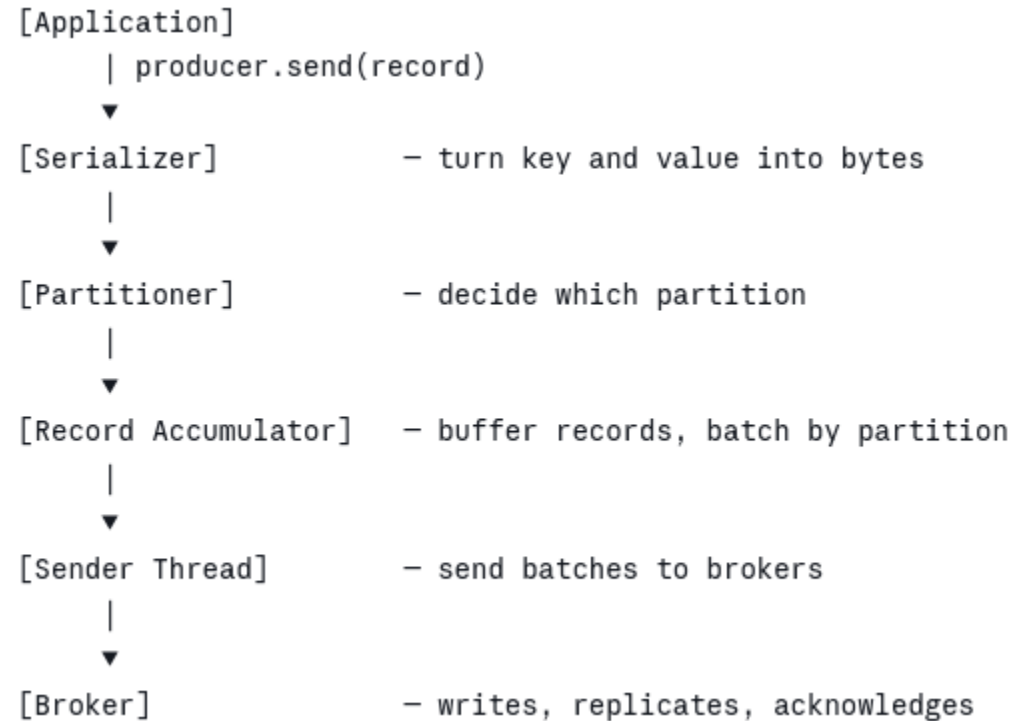
- Cluster metadata storage
 - Kafka used ZooKeeper for coordination:
 - ZooKeeper stored cluster metadata
 - ZooKeeper hosted the controller election
 - Required running a separate ZooKeeper ensemble alongside Kafka
 - Modern Kafka uses KRaft (Kafka Raft):
 - Metadata stored in Kafka itself, as a replicated log
 - Controller election handled by a Raft consensus protocol
 - No ZooKeeper required
 - KRaft is the default for new clusters
 - Both are commonly found in production systems



Producers and Consumers

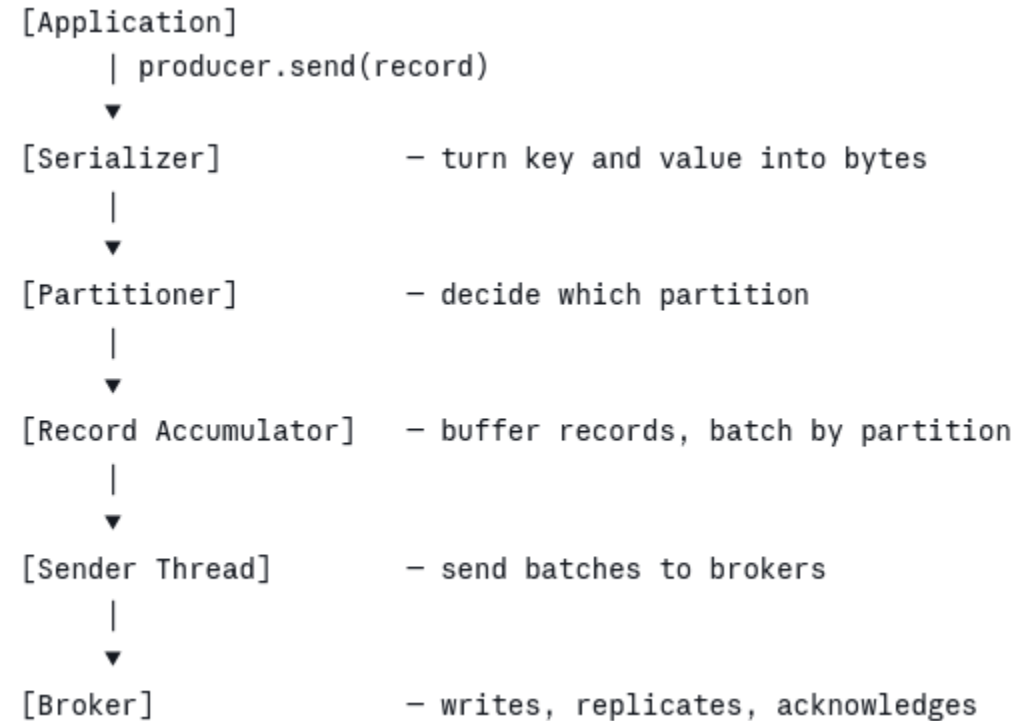
The Producer

- Kafka supplies a Producer API
 - Defines an internal process pipeline
 - When the application calls `send(record)`, the producer
 - Serializes the key and value using the configured serializers
 - The partitioner decides which partition the record should land in
 - The record is appended to an in-memory buffer called the record accumulator, organized by partition
 - A separate sender thread continuously drains this buffer, batching records together by destination broker, and sends them over the network.
 - The application-facing `send()` call returns as soon as the record is accepted into the buffer
 - Returns a Future
 - Actual acknowledgement from the broker comes later, and you retrieve it through the future



The Producer

- Producer API parameters
 - Batching: grouping records going to the same partition into a single request
 - Key setting: batch.size (default 16 KB)
 - Linger: waiting briefly to let more records accumulate before sending
 - Key setting: linger.ms (default 0)
 - Compression: compressing the batch before sending
 - Key setting: compression.type (none, gzip, snappy, lz4, zstd)
 - Larger batches gives better compression ratios
 - Higher linger means larger batches which means higher throughput but higher latency
 - Compression means less network, less storage, more CPU



Send Modes

- The producer's `send()` method
 - Is asynchronous under the hood
 - It returns a `CompletableFuture` regardless of how you use it
 - The "mode" is really about what you do with that future
 - Fire-and-forget means you don't look at it
 - You accept that you won't notice failures
 - This is appropriate for streams where occasional loss is acceptable and the volume makes per-record handling impractical
 - For example, telemetry, metrics, application logs

Mode	How	When to Use
Fire-and-forget	<code>kafkaTemplate.send(topic, key, value)</code> and ignore the returned future	High-volume, loss-tolerant streams (telemetry, metrics, logs)
Asynchronous	<code>send(...)</code> and attach a <code>whenComplete()</code> callback to the returned <code>CompletableFuture</code>	Default for most production code — get delivery confirmation without blocking
Synchronous	<code>send(...).get()</code> to block the calling thread until the broker acknowledges	When the application genuinely cannot proceed until the event is durable

Send Modes

- The producer's `send()` method
 - Asynchronous is the workhorse pattern for production code
 - You attach a callback so failures get logged, alerted on, or routed to a recovery mechanism
 - But the calling thread is never blocked
 - The producer pipeline keeps batching, the application keeps doing work, and you still find out when something goes wrong.
 - Synchronous is implemented by calling `.get()` on the future
 - Blocks the calling thread until the broker returns a response
 - Largely defeats Kafka's batching advantage
 - Use it only when the business semantics require it
 - For example, when a user-facing operation must not return success until the event is durably stored.

```
// Fire-and-forget
kafkaTemplate.send("orders.placed", order.id(), event);

// Asynchronous (recommended default)
kafkaTemplate.send("orders.placed", order.id(), event)
    .whenComplete((result, ex) -> {
        if (ex != null) log.error("Send failed", ex);
    });

// Synchronous
SendResult<String, OrderEvent> result =
    kafkaTemplate.send("orders.placed", order.id(), event).get();
```

ACKS

- The acks setting

- Controls how many brokers must confirm a write before the producer considers it successful.
- acks=0
 - The producer doesn't wait for any acknowledgment
 - It sends the bytes and moves on
 - If the broker rejects th
- acks=1
 - The producer waits for the partition leader to write the record to its own log before acknowledging
 - This catches most failure modes like broker rejections, network errors, serialization issues
 - But if the leader crashes after writing locally but before any follower has replicated the record, the record is lost when a follower is promoted

Level	Who Must Acknowledge	Durability	Latency	When to Use
acks=0	Nobody — fire and forget at the wire	Lowest — silent loss possible	Lowest	High-volume, loss-tolerant streams (telemetry, metrics)
acks=1	The partition leader only	Medium — survives most failures, loses data if leader crashes before replication	Medium	Throughput-sensitive workloads where some loss is acceptable
acks=all (or -1)	Leader + all in-sync replicas	Highest — survives any single broker failure	Highest	Important business data; the production default

ACKS

- The acks setting

- acks=all

- The producer waits for the leader and all in-sync replicas to persist the record

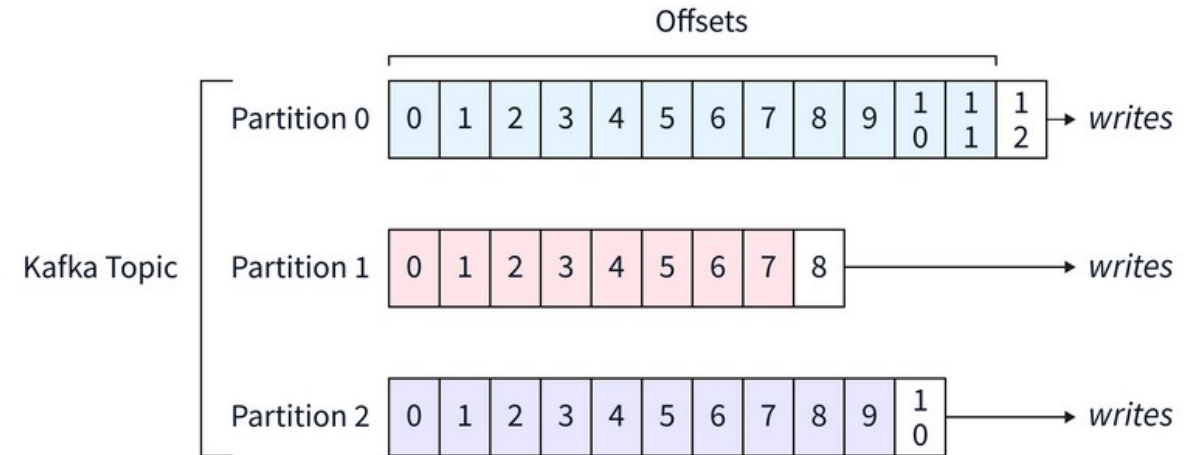
- The latency cost grows with each level

- acks=all requires waiting for replication, which adds milliseconds depending on the cluster
 - For most enterprise scenarios, that cost is well worth the guarantee
 - Modern Kafka makes acks=all with idempotence the default precisely because it's the optimal answer for almost all use cases

Level	Who Must Acknowledge	Durability	Latency	When to Use
<code>acks=0</code>	Nobody — fire and forget at the wire	Lowest — silent loss possible	Lowest	High-volume, loss-tolerant streams (telemetry, metrics)
<code>acks=1</code>	The partition leader only	Medium — survives most failures, loses data if leader crashes before replication	Medium	Throughput-sensitive workloads where some loss is acceptable
<code>acks=all</code> (or <code>-1</code>)	Leader + all in-sync replicas	Highest — survives any single broker failure	Highest	Important business data; the production default

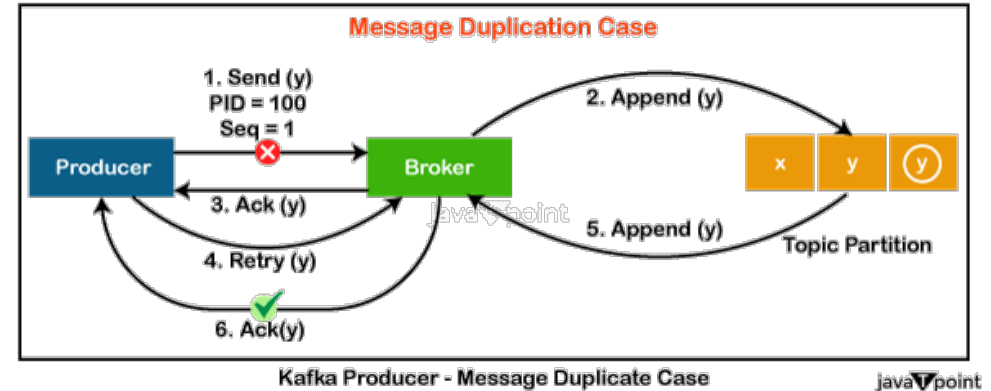
Partitioning Strategies

- Assigning records to partitions
 - If the application specifies a partition explicitly, then use it
 - Else if the record has a key
 - Hash the key, modulo partition count
 - Same key maps to the same partition
 - Preserves ordering for events with the same key
 - Else (no key)
 - Pick one partition, send records to it until the batch fills, then switch
 - Replaces older round-robin to improve batching efficiency
 - Custom partitioners are possible but rarely needed



Idempotent Producers

- Network failures can cause producers to retry
 - Without checking, a retry might cause a duplicate:
 - Original request succeeded
 - Acknowledgement was lost
 - Producer retries
 - Broker receives the same event twice
 - Idempotent producers eliminate this:
 - Each producer is assigned a unique producer ID
 - Each message gets a per-partition sequence number
 - Broker rejects duplicate sequence numbers
 - Default in modern Kafka



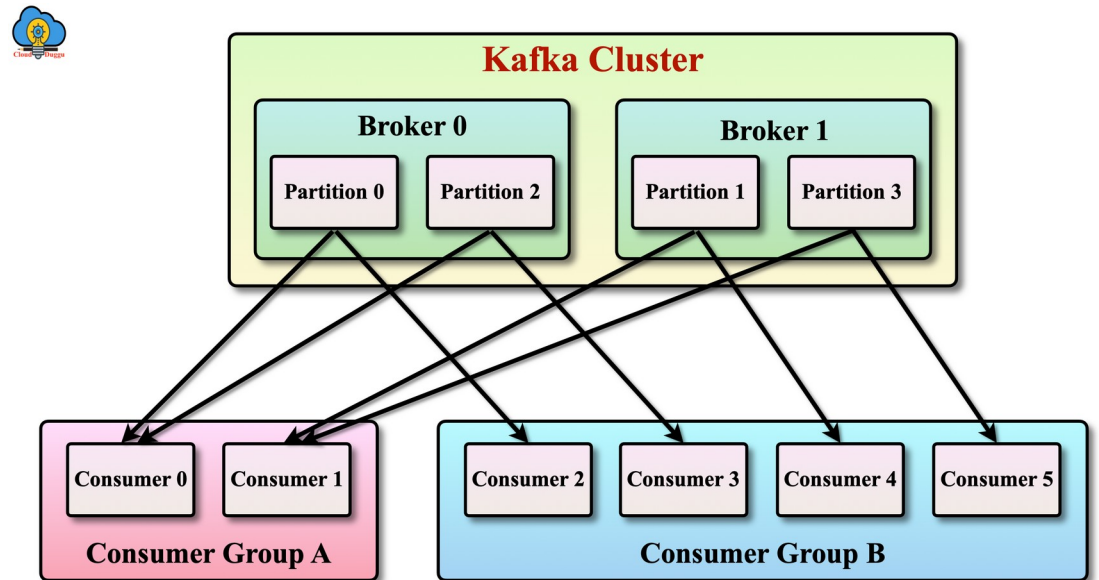
Consumer

- Built around a single polling loop
 - Every poll() call:
 - Sends heartbeats to the group coordinator
 - Fetches events from assigned partitions
 - Triggers rebalance handling if needed
 - Returns a batch of records to the application
 - The poll call is also the consumer's liveness signal
 - Failure to poll over a timeout period
 - Interpreted by the Broker that the consumer has terminated
 - Triggers a rebalance
 - The dead consumer's partitions are assigned to another consumer in the same consumer group

```
while (running):  
    records = consumer.poll(timeout)  
    process(records)  
    consumer.commitSync()    // or async
```

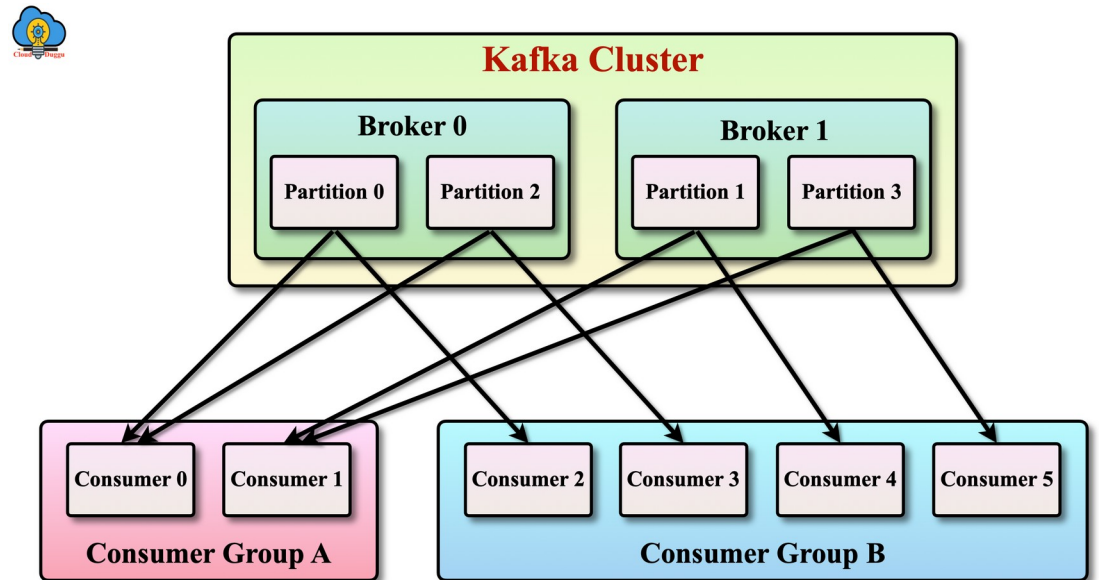
Consumer Groups

- Consumers in the same group share the topic's partitions
 - Kafka assigns each partition to exactly one consumer in the group
 - Assignment is done the Kafka lead broker
- Assignment strategies
 - Range (default): contiguous partitions per consumer
 - Round-robin: alternating partitions across consumers
 - Sticky: minimize movement on rebalance



Consumer Groups Rebalance

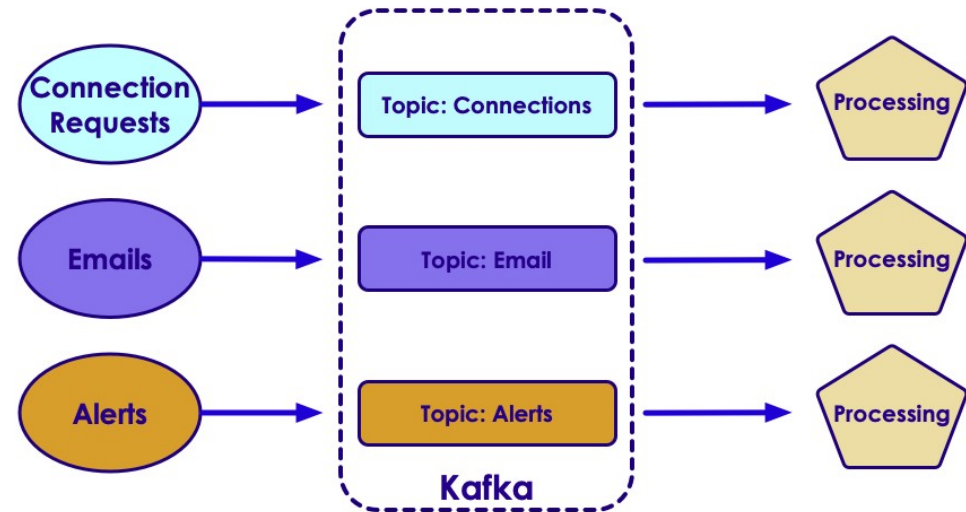
- A rebalance redistributes partitions across consumer group members
 - Triggered by
 - A new consumer joining the group
 - An existing consumer leaving (gracefully or via failure)
 - A change in topic subscription
 - Partition count increasing for a subscribed topic
 - Classic rebalance ("eager" rebalance)
 - All consumers stop processing
 - New assignment is computed
 - All consumers resume with their new partitions
 - Cooperative rebalance
 - Only the partitions being reassigned are paused
 - Other processing continues



Design Considerations

Topic Design

- Topic Ownership
 - In a multi-team organization, topics without clear ownership become liabilities
 - Ownership means
 - This team decides the event schema
 - This team is responsible for schema evolution
 - This team is on call for issues with this topic
 - Consumers know whom to contact for changes or clarifications
 - Document ownership in a topic registry or data catalog
 - Without ownership, you get "topic rot"
 - Stale message contracts nobody will fix



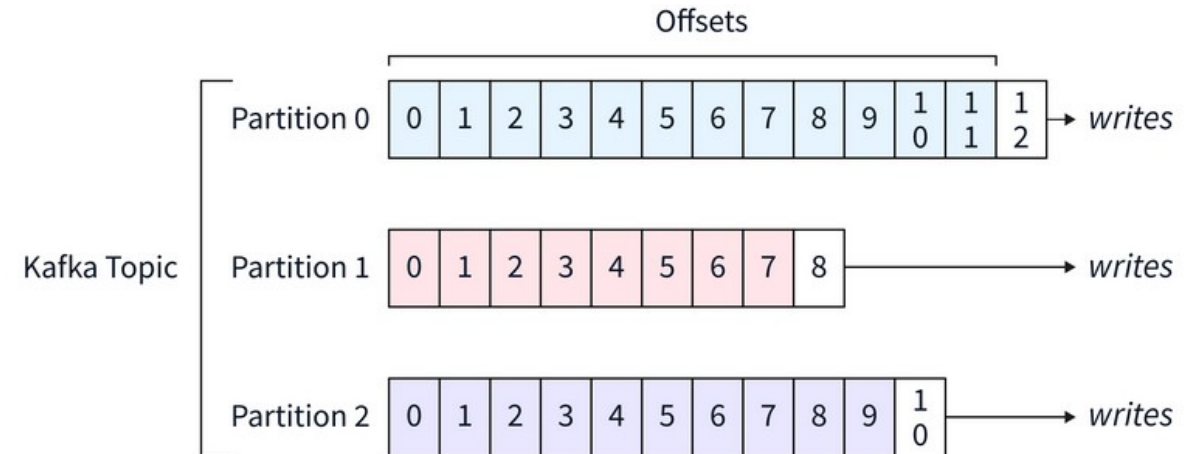
Topic Design

- Costs of badly designed topics.
 - Renaming a topic requires consumer migration
 - Changing partition count disrupts key-based ordering
 - Changing cleanup policy between delete and compact means complicated migration
 - Changing replication factor mean cluster-level operation with performance impact
- Design topics as if they were APIs, because they are

Setting	What It Controls
<code>partitions</code>	Parallelism and ordering scope
<code>replication.factor</code>	Durability and fault tolerance
<code>retention.ms</code>	How long events are kept (time-based)
<code>retention.bytes</code>	How large the partition can grow before cleanup
<code>cleanup.policy</code>	<code>delete</code> (default) or <code>compact</code>
<code>min.insync.replicas</code>	Minimum replicas that must ack a write
<code>compression.type</code>	Broker-level compression policy

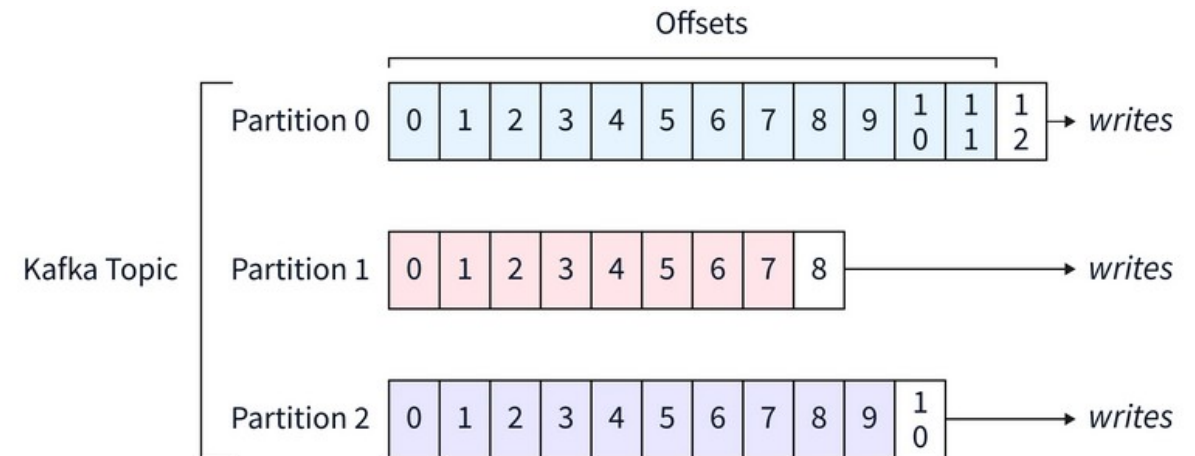
Partitioning Strategy

- What partitioning controls
 - Parallelism
 - More partitions = more parallel consumers possible
 - Ordering
 - Ordering is preserved within a partition, not across
 - Distribution
 - Partitions spread storage and load across brokers
 - These three concerns sometimes pull in different directions
 - Partition count and key selection are how you balance them



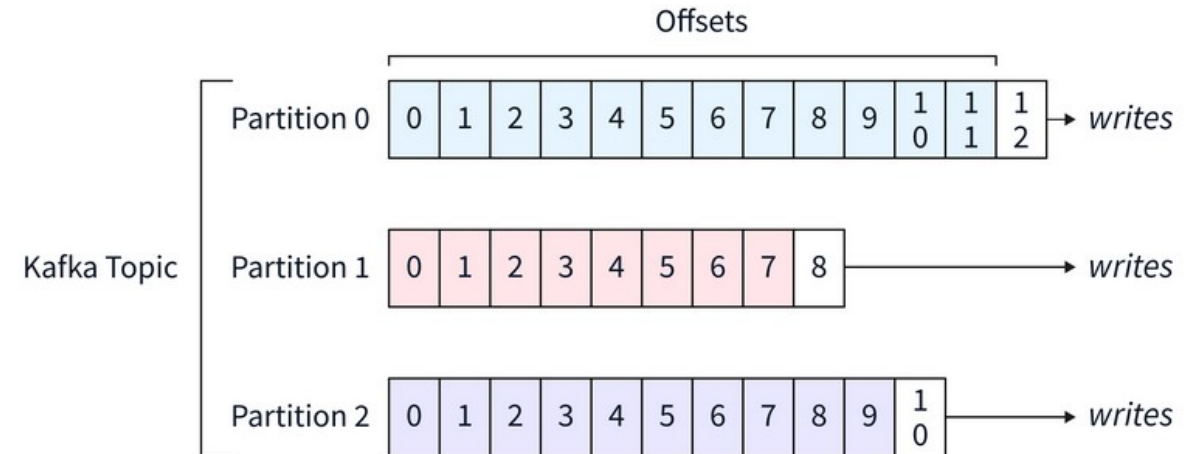
Partitioning Strategy

- Choosing a partition count
 - Partition count is a capacity and parallelism decision
 - Lower bounds
 - At least as many as the maximum expected consumer count per group
 - Enough to distribute load across brokers
 - Upper bounds
 - More partitions = more file handles, more memory, longer rebalances
 - More partitions = more metadata for the cluster to manage
 - Rule of thumb: start with enough partitions to cover expected peak parallelism, plus some headroom



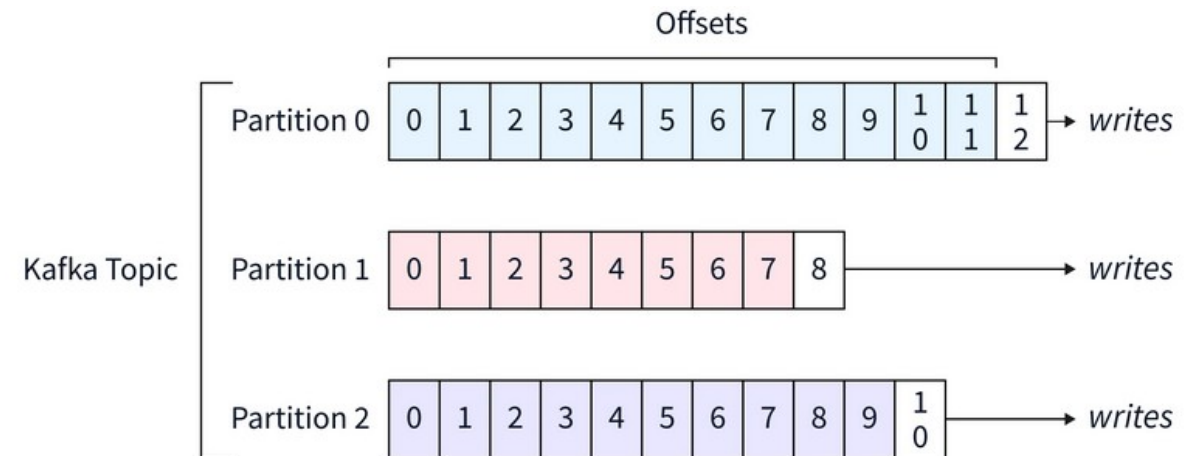
Partitioning Strategy

- Changing a partition count
 - You can add partitions to an existing topic
 - But the key-to-partition mapping changes
 - Events with key "customer-42" that used to go to partition 3 might now go to partition 7
 - Historical events remain in their original partitions
 - Consumers that depend on per-key ordering break silently
 - You cannot reduce partition count at all



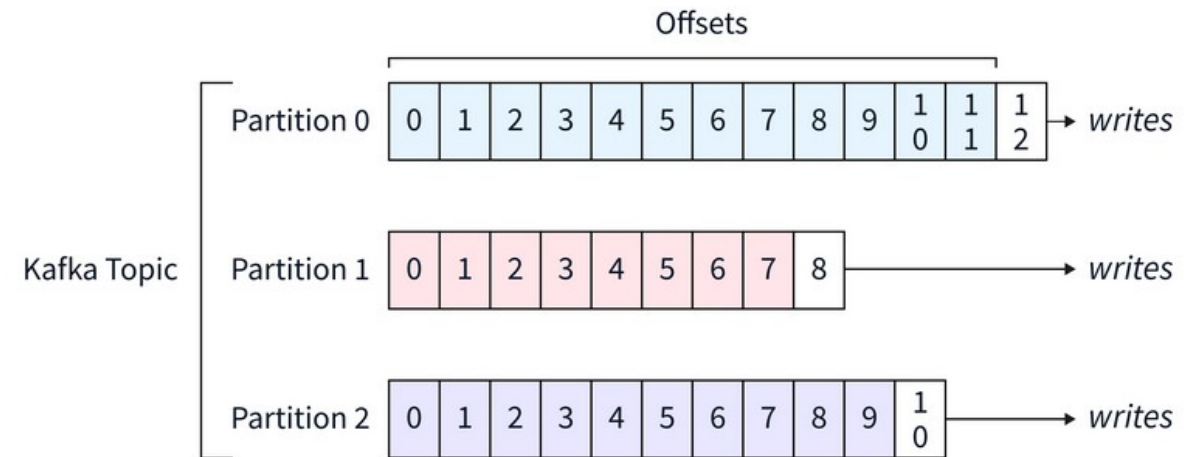
Partitioning Strategy

- Choosing a partition key
 - The key determines which partition an event goes to
- Events with the same key are ordered relative to each other
- Events with different keys have no ordering guarantee
- Good key = the "thing" whose lifecycle you want ordered:
 - For order events: the order ID
 - For user activity: the user ID
 - For account transactions: the account ID
- Bad keys are often too granular or too coarse



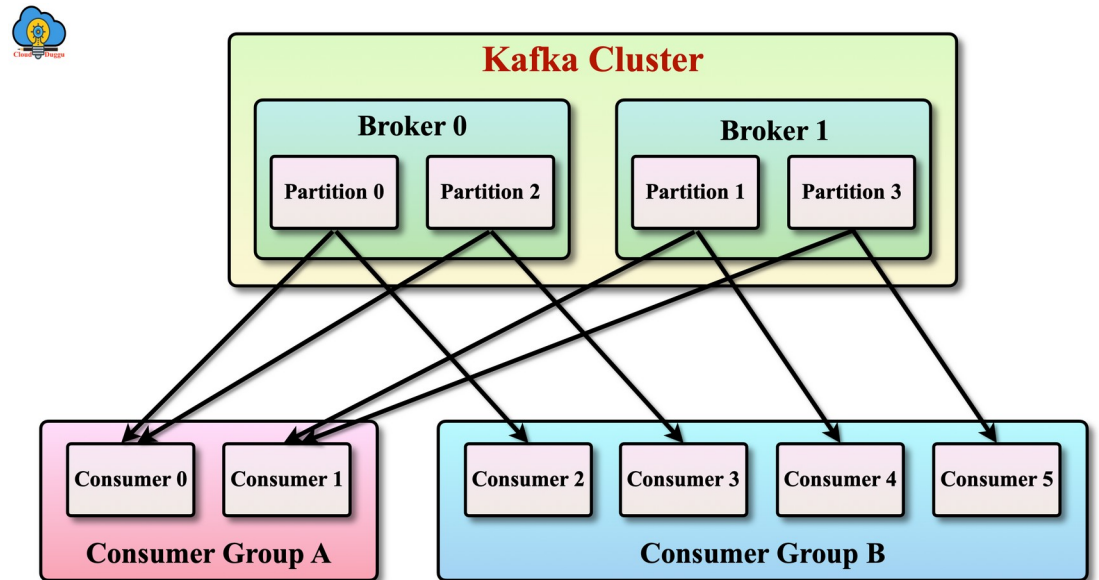
Partitioning Strategy

- When to skip the key
 - Not every event needs a key
 - No key means the producer uses a partitioning algorithm
- Good fit when
 - Events are independent of each other
 - No per-entity ordering is required
 - Maximum throughput matters more than ordering
 - Examples
 - Log streams
 - Metrics samples
 - Clickstream events



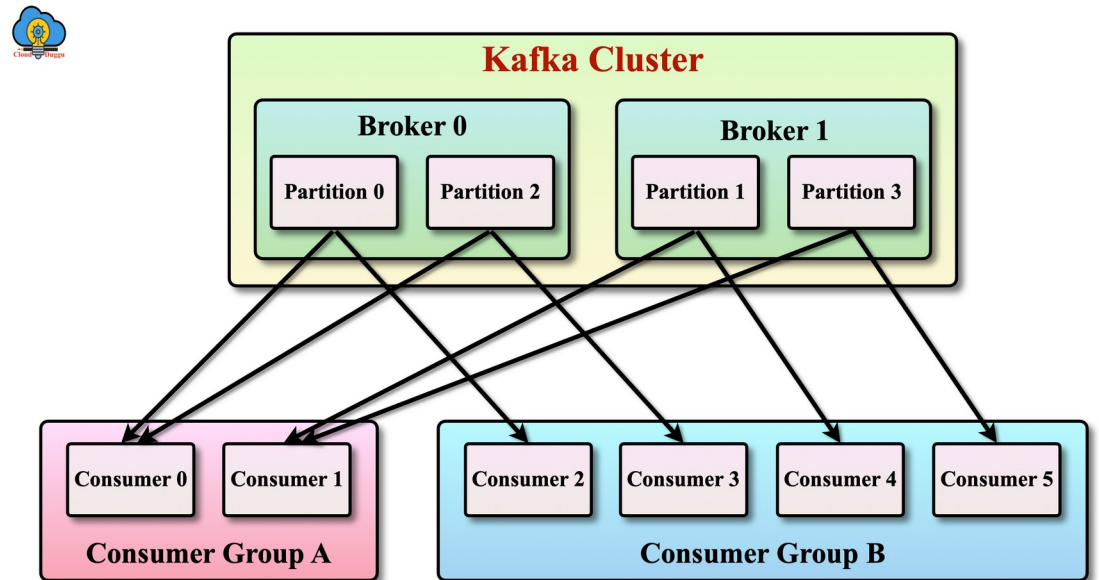
Consumer Groups

- The consumer group rule
 - One partition, one consumer (per group)
 - Within a consumer group, Kafka assigns each partition to exactly one consumer
 - Parallelism within a group is capped at the partition count
 - Adding a 13th consumer to a group reading a 12-partition topic means the 13th sits idle
 - Across groups, no such constraint
 - Each group gets its own copies of the messages



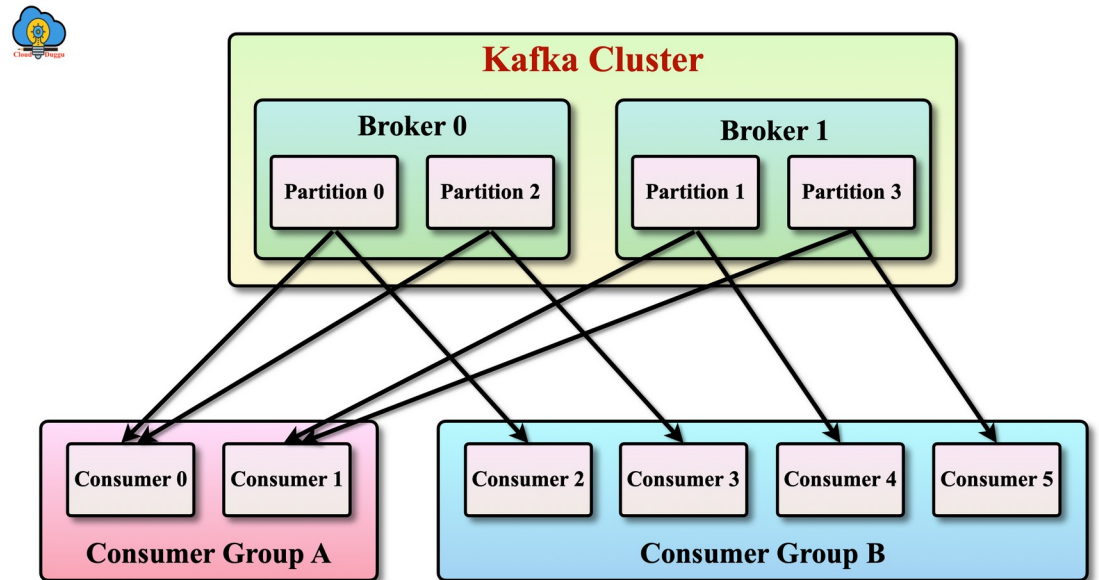
Consumer Groups

- Sizing a consumer group
 - Main question: what's the processing rate you need vs. the production rate?
 - Factors
 - Peak event arrival rate
 - Average processing time per event
 - Acceptable consumer lag
 - Start point
 - $\text{Required_consumers} = (\text{peak_rate} \times \text{avg_processing_time}) / \text{target_utilization}$
 - Then cap at partition count



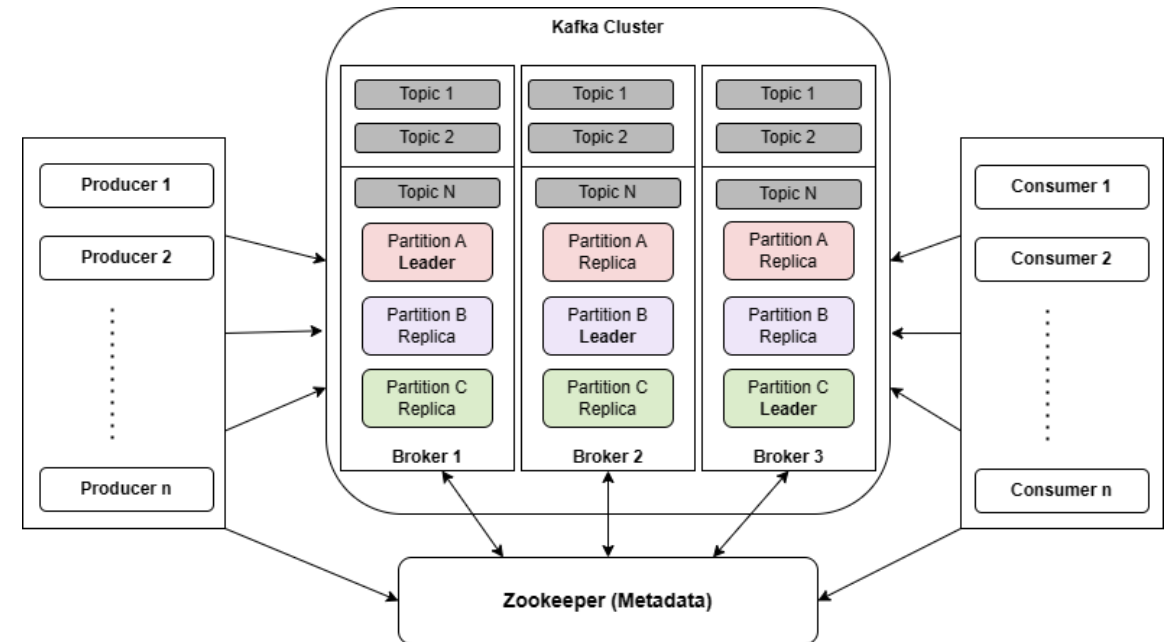
Consumer Groups

- Number of consumer groups
 - A single topic can support many consumer groups simultaneously
 - Each group has its own
 - Offset positions
 - Membership and coordination
 - Processing logic
 - Groups are a fan-out
 - Each group represents a domain of concern
 - Orders topic might be read by independent interests
 - Inventory services
 - Accounting services
 - Warehouse management services
 - Each interest would have its own consumer group
 - New consumer groups are additive and have zero impact on existing ones



Topic Design Checklist

- When designing a topic
 - Topic name follows the naming convention
 - Topic ownership is documented
 - Granularity (per event, per aggregate, per context) is deliberate
 - Partition count accounts for peak parallelism and growth
 - Partition key is chosen with ordering requirements in mind
 - Known hot-key risks have been considered
 - Retention matches consumer replay needs
 - Replication factor and min.insync.replicas are set for durability requirements
 - Consumer groups and their purposes are enumerated



Questions?

